

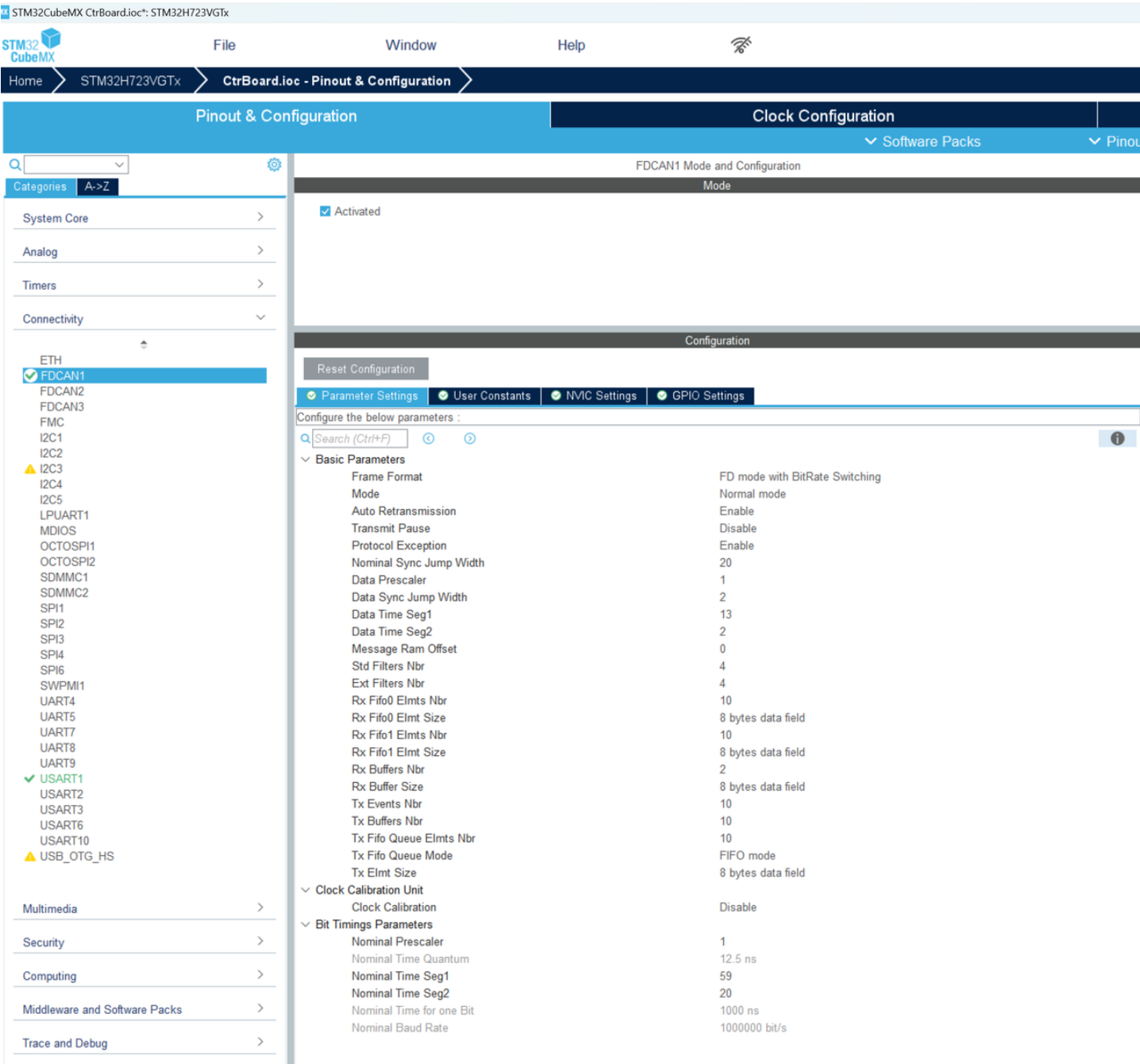
2. 达妙电机上手流程(二)-使用开发板控制电机

1. FDCAN和CAN协议介绍

2. STM32CubeMX 配置参数详细说明

2.1 STM32CubeMx 配置界面：

以 DM-MC02(H723) 为例



2.2 各配置参数详细说明

- **Frame Format**: 用于设置CAN帧格式

Code block

```
1  #define FDCAN_FRAME_CLASSIC ((uint32_t)0x00000000U) /* 经典 CAN 模式 */
2  #define FDCAN_FRAME_FD_NO_BRS ((uint32_t)FDCAN_CCCR_FDOE) /* FD CAN 不带可变波特率 */
3  #define FDCAN_FRAME_FD_BRS ((uint32_t)(FDCAN_CCCR_FDOE | FDCAN_CCCR_BRSE)) /* FD CAN 带可变波特率 */
```

- **Mode**: 用于设置CAN的操作模式

Code block

```
1  #define FDCAN_MODE_NORMAL ((uint32_t)0x00000000U) /*!< 正常模式 */
2  #define FDCAN_MODE_RESTRICTED_OPERATION ((uint32_t)0x00000001U) /*!< 有限的操作模式 */
3  #define FDCAN_MODE_BUS_MONITORING ((uint32_t)0x00000002U) /*!< 总线监测模式 */
4  #define FDCAN_MODE_INTERNAL_LOOPBACK ((uint32_t)0x00000003U) /*!< 内部环回模式 */
5  #define FDCAN_MODE_EXTERNAL_LOOPBACK ((uint32_t)0x00000004U) /*!< 外部环回模式 */
```

- **AutoRetransmission**: 自动重发功能, 使能 Enable 或者禁止 Disable
- **TransmitPause**: 传输暂停功能, 使能 Enable 或者禁止 Disable
- **ProtocolException**: 协议异常管理功能, 使能 Enable 或者禁止 Disable
- **NominalPrescaler**: 用于 CAN FD 仲裁阶段分频设置, 范围 1-512
- **NominalSyncJumpWidth**: 设置 FD CAN 仲裁阶段最大支持的时间量来加长或者缩短一个 bit 来实现再同步, 范围 1-128
- **NominalTimeSeg1**: 设置仲裁阶段 Bit Segment 1 的时间量, 范围 2-256
- **NominalTimeSeg2**: 设置仲裁阶段 Bit Segment 2 的时间量, 范围 2-128
- **DataPrescaler**: 用于 CAN FD 数据阶段分频设置, 范围 1-32
- **DataSyncJumpWidth**: 设置FD CAN数据阶段最大支持的时间量来加长或者缩短一个bit来实现数据再同步, 参数范围1-16
- **DataTimeSeg1**: 设置数据阶段 Data Bit Segment 1 的时间量, 范围 1 – 32。
- **DataTimeSeg2**: 设置数据阶段 Data Bit Segment 2 的时间量, 范围 1 – 16。
- **MessageRAMOffset**: 设置消息 RAM 起始地址, 范围 0 到 2560。
- **StdFiltersNbr**: 标准 ID 过滤个数, 范围 0 到 128。

- **ExtFiltersNbr**: 扩展 ID 过滤个数, 范围 0 到 64。
- **RxFifo0ElmtsNbr**: RX FIFO0 元素个数, 范围 0 到 64。
- **RxFifo0ElmtSize**: RX FIFO0 每个元素中数据大小, 支持参数如下:

Code block

```
1  #define FDCAN_DATA_BYTES_8 ((uint32_t)0x00000004U) /*!< 8 bytes data field */
2  #define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */

3  #define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */

4  #define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */

5  #define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */

6  #define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */

7  #define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */

8  #define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */
```

- **RxFifo1ElmtsNbr**: RX FIFO1 个数, 范围 0 到 64。
- **RxFifo1ElmtSize**: RX FIFO1 每个元素中数据大小, 支持参数如下:

Code block

```
1  #define FDCAN_DATA_BYTES_8 ((uint32_t)0x00000004U) /*!< 8 bytes data field */
2  #define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */

3  #define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */

4  #define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */

5  #define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */

6  #define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */

7  #define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */

8  #define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */
```

- **RxBuffersNbr**: 设置 Rx Buffer 元素个数, 范围 0 - 64
- **RxBuffersSize**: 设置 Rx Buffer 元素中每个数据大小, 范围 0 - 64:

```

1  #define FDCAN_DATA_BYTES_8 ((uint32_t)0x00000004U) /*!< 8 bytes data field */
2  #define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */

3  #define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */

4  #define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */

5  #define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */

6  #define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */

7  #define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */

8  #define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */

```

- **TxEventsNbr**: Tx Event FIFO 元素个数，范围 0 到 32。
- **TxBuffersNbr**: 设置专用的 Tx Buffer 元素个数，范围 0 到 32。
- **TxFifoQueueElmtsNbr**: 设置用于 Tx FIFO/Queue 的 Tx Buffers 个数。范围 0 到 32。
- **TxFifoQueueMode**: 设置 FIFO 模式或者 QUEUE 队列模式。

Code block

```

1  #define FDCAN_TX_FIFO_OPERATION ((uint32_t)0x00000000U) /*!< FIFO mode */
2  #define FDCAN_TX_QUEUE_OPERATION ((uint32_t)FDCAN_TXBC_TFQM) /*!< Queue mode */

```

- **TxEltSize**: 设置 Tx Element 中的数据域大小。支持参数如下:

Code block

```

1  #define FDCAN_DATA_BYTES_8 ((uint32_t)0x00000004U) /*!< 8 bytes data field */
2  #define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */

3  #define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */

4  #define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */

5  #define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */

6  #define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */

7  #define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */

8  #define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */

```

3. FDCAN 波特率和采样点

3.1 FDCAN 波特率和采样点计算

- 仲裁段波特率计算，假设CAN时钟为80M

Code block

```
1 nominal_baud = 80M/NominalPrescaler/(1+NominalTimeSeg1+NominalTimeSeg2)
2 sample_point = (1+NominalTimeSeg1)/(1+NominalTimeSeg1+NominalSyncJumpWidth)
```

- 数据段波特率计算，假设CAN时钟为80M

Code block

```
1 data_baud = 80M/DataPrescaler/(1+DataTimeSeg1+DataTimeSeg2)
2 sample_point = (1+DataTimeSeg1)/(1+DataTimeSeg1+DataSyncJumpWidth)
```

3.2 FDCAN波特率推荐设置

- BRP应尽量小以保证Tg尽可能小，减少误差
- CANFD推荐仲裁域BRP<=数据域BRP
- SJW应尽量大，尽量保持与TSEG2一致，以提高位宽容忍度;
- 采样点推荐范围:75~87.5%

波特率>800kbps时，推荐采样点75%

波特率>500kbps时，推荐采样点80%

波特率<= 500kbps时推荐采样点87.5%

- 应尽量保证总线上所有节点采样点一致，CANFD仲裁域和数据域采样点不要求一致
- CANFD仲裁域与数据域波特率之比应大于1/8

3.3 CAN 波特率计算小工具



BitrateTool.exe

5.12MB



3.4 CAN 波特率设置bsp代码

Code block

```
1 #define CAN_CLASS 0 // 经典CAN
```

```

2  #define CAN_FD_BRS 1    // FDCAN
3
4  #define CAN_BR_125K 0
5  #define CAN_BR_200K 1
6  #define CAN_BR_250K 2
7  #define CAN_BR_500K 3
8  #define CAN_BR_1M 4
9  #define CAN_BR_2M 5
10 #define CAN_BR_2M5 6
11 #define CAN_BR_3M2 7
12 #define CAN_BR_4M 8
13 #define CAN_BR_5M 9
14
15 void bsp_fdcan_set_baud(hcan_t *hfdcan, uint8_t mode, uint8_t baud)
16 {
17     uint32_t nom_brp=0, nom_seg1=0, nom_seg2=0, nom_sjw=0;
18     uint32_t dat_brp=0, dat_seg1=0, dat_seg2=0, dat_sjw=0;
19
20     /*      nominal_baud = 80M/brp/(1+seg1+seg2)
21            sample point = (1+seg1)/(1+seg1+sjw) */
22     if(mode == CAN_CLASS)
23     {
24         switch (baud)
25         {
26             case CAN_BR_125K:      nom_brp=4 ; nom_seg1=139; nom_seg2=20;
nom_sjw=20; break; // sample point 87.5%
27             case CAN_BR_200K:      nom_brp=2 ; nom_seg1=174; nom_seg2=25;
nom_sjw=25; break; // sample point 87.5%
28             case CAN_BR_250K:      nom_brp=2 ; nom_seg1=139; nom_seg2=20;
nom_sjw=20; break; // sample point 87.5%
29             case CAN_BR_500K:      nom_brp=1 ; nom_seg1=139; nom_seg2=20;
nom_sjw=20; break; // sample point 87.5%
30             case CAN_BR_1M:        nom_brp=1 ; nom_seg1=59 ; nom_seg2=20;
nom_sjw=20; break; // sample point 75%
31         }
32         dat_brp=1 ; dat_seg1=29; dat_seg2=10; dat_sjw=10; // 数据域默认1M
33         hfdcan->Init.FrameFormat = FDCAN_FRAME_CLASSIC;
34     }
35     /*      data_baud      = 80M/brp/(1+seg1+seg2)
36            sample point = (1+seg1)/(1+seg1+sjw) */
37     if(mode == CAN_FD_BRS)
38     {
39         switch (baud)
40         {
41             case CAN_BR_2M:        dat_brp=1 ; dat_seg1=29; dat_seg2=10;
dat_sjw=10; break; // sample point 75%

```

```

42         case CAN_BR_2M5:          dat_brp=1 ; dat_seg1=25; dat_seg2=6 ;
dat_sjw=6 ; break;                // sample point 81.25%
43         case CAN_BR_3M2:          dat_brp=1 ; dat_seg1=19; dat_seg2=5 ;
dat_sjw=5 ; break;                // sample point 80%
44         case CAN_BR_4M:           dat_brp=1 ; dat_seg1=14; dat_seg2=5 ;
dat_sjw=5 ; break;                // sample point 75%
45         case CAN_BR_5M:           dat_brp=1 ; dat_seg1=13; dat_seg2=2 ;
dat_sjw=2 ; break;                // sample point 87.5%
46     }
47     nom_brp=1 ; nom_seg1=59 ; nom_seg2=20; nom_sjw=20; // 仲裁域默认1M
48     hfdcan->Init.FrameFormat = FDCAN_FRAME_FD_BRS;
49 }
50
51 HAL_FDCAN_DeInit(hfdcan);
52
53 hfdcan->Init.NominalPrescaler = nom_brp;
54 hfdcan->Init.NominalTimeSeg1  = nom_seg1;
55 hfdcan->Init.NominalTimeSeg2  = nom_seg2;
56 hfdcan->Init.NominalSyncJumpWidth = nom_sjw;
57
58 hfdcan->Init.DataPrescaler = dat_brp;
59 hfdcan->Init.DataTimeSeg1  = dat_seg1;
60 hfdcan->Init.DataTimeSeg2  = dat_seg2;
61 hfdcan->Init.DataSyncJumpWidth = dat_sjw;
62
63 HAL_FDCAN_Init(hfdcan);
64 }

```

4. FDCAN 滤波器

4.1 滤波器结构体代码

Code block

```

1  typedef struct
2  {
3      uint32_t IdType;          /*!< Specifies the identifier type.
4                                  This parameter can be a value of @ref
5                                  FDCAN_id_type */
6      uint32_t FilterIndex;     /*!< Specifies the filter which will be
7                                  initialized.
8                                  This parameter must be a number between:
9                                  - 0 and 127, if IdType is FDCAN_STANDARD_ID

```

```

9          - 0 and 63, if IdType is FDCAN_EXTENDED_ID
          */
10
11      uint32_t FilterType;      /*!< Specifies the filter type.
12                                This parameter can be a value of @ref
13                                FDCAN_filter_type.
14                                The value FDCAN_EXT_FILTER_RANGE_NO_EIDM is
15                                permitted
16                                only when IdType is FDCAN_EXTENDED_ID.
17                                This parameter is ignored if FilterConfig is
18                                set to
19                                FDCAN_FILTER_TO_RXBUFFER
20                                */
21      uint32_t FilterConfig;    /*!< Specifies the filter configuration.
22                                This parameter can be a value of @ref
23                                FDCAN_filter_config */
24
25      uint32_t FilterID1;       /*!< Specifies the filter identification 1.
26                                This parameter must be a number between:
27                                - 0 and 0x7FF, if IdType is
28                                FDCAN_STANDARD_ID
29                                - 0 and 0x1FFFFFFF, if IdType is
30                                FDCAN_EXTENDED_ID */
31
32      uint32_t FilterID2;       /*!< Specifies the filter identification 2.
33                                This parameter is ignored if FilterConfig is
34                                set to
35                                FDCAN_FILTER_TO_RXBUFFER.
36                                This parameter must be a number between:
37                                - 0 and 0x7FF, if IdType is
38                                FDCAN_STANDARD_ID
39                                - 0 and 0x1FFFFFFF, if IdType is
40                                FDCAN_EXTENDED_ID */
41
42      uint32_t RxBufferIndex;    /*!< Contains the index of the Rx buffer in
43                                which the
44                                matching message will be stored.
45                                This parameter must be a number between 0
46                                and 63.
47                                This parameter is ignored if FilterConfig is
48                                different
49                                from FDCAN_FILTER_TO_RXBUFFER
50                                */
51
52      uint32_t IsCalibrationMsg; /*!< Specifies whether the filter is configured
53                                for

```



```

40         calibration messages.
41         This parameter is ignored if FilterConfig is
           different
42         from FDCAN_FILTER_TO_RXBUFFER.
43         This parameter can be:
44         - 0 : ordinary message
45         - 1 : calibration message
           */
46 } FDCAN_FilterTypeDef;

```

- **IdType**: 用于设置标准 ID 和扩展 ID。

Code block

```

1  #define FDCAN_STANDARD_ID ((uint32_t)0x00000000U) /*!< 标准 ID */
2  #define FDCAN_EXTENDED_ID ((uint32_t)0x40000000U) /*!< 扩展 ID */

```

- **FilterIndex**: 用于过滤索引，如果是标准 ID，范围 0 到 127。如果是扩展 ID，范围 0 到 64。
- **FilterType**: 用于设置过滤类型。如果成员 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER，本参数将不起作用。

Code block

```

1  #define FDCAN_FILTER_RANGE ((uint32_t)0x00000000U) /*!< 范围过滤从 FilterID1 到
           FilterID2 */
2  #define FDCAN_FILTER_DUAL ((uint32_t)0x00000001U) /*!< 专用 ID 过滤, FilterID1 或
           者 FilterID2 */
3  /*!< 精度屏蔽过滤, FilterID1 = filter, FilterID2 = mask */
4  #define FDCAN_FILTER_MASK ((uint32_t)0x00000002U)
5  /*!< 仅 ID 扩展模式支持此参数, 范围从 FilterID1 到 FilterID2, EIDM mask not
           applied */
6  #define FDCAN_FILTER_RANGE_NO_EIDM ((uint32_t)0x00000003U)

```

- **FilterConfig**: 用于设置过滤类型。

Code block

```

1  #define FDCAN_FILTER_DISABLE ((uint32_t)0x00000000U) //禁止过滤
2  #define FDCAN_FILTER_TO_RXFIFO0 ((uint32_t)0x00000001U) //如果过滤匹配, 将数据保存
           到 Rx FIFO 0
3  #define FDCAN_FILTER_TO_RXFIFO1 ((uint32_t)0x00000002U) //如果过滤匹配, 将数据保存
           到 Rx FIFO 1
4  #define FDCAN_FILTER_REJECT ((uint32_t)0x00000003U) //如果过滤匹配, 拒绝此 ID
5  #define FDCAN_FILTER_HP ((uint32_t)0x00000004U) //如果过滤匹配, 设置高优先级

```

```

6  #define FDCAN_FILTER_TO_RXFIFO0_HP ((uint32_t)0x00000005U) //如果过滤匹配, 设置高
    优先级并保存到 FIFO 0
7  #define FDCAN_FILTER_TO_RXFIFO1_HP ((uint32_t)0x00000006U) //如果过滤匹配, 设置高
    优先级并保存到 FIFO 1
8  #define FDCAN_FILTER_TO_RXBUFFER ((uint32_t)0x00000007U) //如果过滤匹配, 保存到
    Rx Buffer, 并忽略 FilterType 配置

```

- **FilterID1**: 用于设置过滤 ID1。如果 ID 类型是 FDCAN_STANDARD_ID, 范围 0 到 0x7FF。如果 ID 类型是 FDCAN_EXTENDED_ID, 范围是 0 到 0x1FFFFFFF。
- **FilterID2**: 用于设置过滤 ID2。如果 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER, 此参数不起作用。如果 ID 类型是 FDCAN_STANDARD_ID, 范围 0 到 0x7FF。如果 ID 类型是 FDCAN_EXTENDED_ID, 范围是 0 到 0x1FFFFFFF。
- **RxBufferIndex**: 匹配消息存储到 Rx buffer 中的索引。参数范围 0 到 63。如果 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER, 此参数不起作用。
- **IsCalibrationMsg**: 用于设置是否配置校准消息。如果 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER, 此参数不起作用。0: 表示正常消息, 1: 表示校准消息。

4.2 滤波器代码初始化

Code block

```

1  // 默认设置不过滤id
2  void can_filter_init(void)
3  {
4      FDCAN_FilterTypeDef fdcan_filter;
5
6      fdcan_filter.IdType = FDCAN_STANDARD_ID;           //标准ID
7      fdcan_filter.FilterIndex = 0;                     //滤波器索引
8
9      fdcan_filter.FilterType = FDCAN_FILTER_MASK;
10     fdcan_filter.FilterConfig = FDCAN_FILTER_TO_RXFIFO0; //过滤器0关联
        到FIFO0
11     fdcan_filter.FilterID1 = 0x00;
12     fdcan_filter.FilterID2 = 0x00;
13
14     HAL_FDCAN_ConfigFilter(&hfdcan1,&fdcan_filter);
        //接收ID2
15     //拒绝接收匹配不成功的标准ID和扩展ID, 不接受远程帧
16     HAL_FDCAN_ConfigGlobalFilter(&hfdcan1,FDCAN_REJECT,FDCAN_REJECT,FDCAN_REJECT_RE
        MOTE,FDCAN_REJECT_REMOTE);
17     HAL_FDCAN_ConfigFifoWatermark(&hfdcan1, FDCAN_CFG_RX_FIFO0, 1);
18 }

```

5. FDCAN 发送消息

5.1 FDCAN 发送消息结构体

Code block

```
1  typedef struct
2  {
3      uint32_t Identifier;          /*!< Specifies the identifier.
4                                     This parameter must be a number between:
5                                     - 0 and 0x7FF, if IdType is
6                                     FDCAN_STANDARD_ID
7                                     - 0 and 0x1FFFFFFF, if IdType is
8                                     FDCAN_EXTENDED_ID */
9      uint32_t IdType;             /*!< Specifies the identifier type for the
10                                     message that will be
11                                     transmitted.
12                                     This parameter can be a value of @ref
13                                     FDCAN_id_type */
14      uint32_t TxFrameType;        /*!< Specifies the frame type of the message
15                                     that will be transmitted.
16                                     This parameter can be a value of @ref
17                                     FDCAN_frame_type */
18      uint32_t DataLength;         /*!< Specifies the length of the frame that
19                                     will be transmitted.
20                                     This parameter can be a value of @ref
21                                     FDCAN_data_length_code */
22      uint32_t ErrorStateIndicator; /*!< Specifies the error state indicator.
23                                     This parameter can be a value of @ref
24                                     FDCAN_error_state_indicator */
25      uint32_t BitRateSwitch;      /*!< Specifies whether the Tx frame will be
26                                     transmitted with or without
27                                     bit rate switching.
28                                     This parameter can be a value of @ref
29                                     FDCAN_bit_rate_switching */
30      uint32_t FDFormat;           /*!< Specifies whether the Tx frame will be
31                                     transmitted in classic or
32                                     FD format.
```

```

27                                     This parameter can be a value of @ref
    FDCAN_format                      */
28
29     uint32_t TxEventFifoControl; /*!< Specifies the event FIFO control.
30                                     This parameter can be a value of @ref
    FDCAN_EFC                        */
31
32     uint32_t MessageMarker; /*!< Specifies the message marker to be
    copied into Tx Event FIFO
33                                     element for identification of Tx message
    status.
34                                     This parameter must be a number between 0
    and 0xFF                        */
35 } FDCAN_TxHeaderTypeDef;

```

- **Identifier**: 用于设置 ID，如果 IdType 是标准 FDCAN_STANDARD_ID，范围 0 到 0x7FF，如果 IdType 是 FDCAN_EXTENDED_ID 扩展 ID，范围 0 到 0x1FFFFFFF。
- **IdType**: 用于设置标准 ID 或者扩展 ID。

Code block

```

1  #define FDCAN_STANDARD_ID ((uint32_t)0x00000000U) /*!< 标准 ID */
2  #define FDCAN_EXTENDED_ID ((uint32_t)0x40000000U) /*!< 扩展 ID */

```

- **TxFrametype**: 用于设置帧类型，数据帧或遥控帧。

Code block

```

1  #define FDCAN_DATA_FRAME ((uint32_t)0x00000000U) /*!< 数据帧 */
2  #define FDCAN_REMOTE_FRAME ((uint32_t)0x20000000U) /*!< 远程帧 */

```

- **DataLength**: 用于设置数据长度。

Code block

```

1  #define FDCAN_DLC_BYTES_0 ((uint32_t)0x00000000U) /*!< 0 bytes data field */
2  #define FDCAN_DLC_BYTES_1 ((uint32_t)0x00010000U) /*!< 1 bytes data field */
3  #define FDCAN_DLC_BYTES_2 ((uint32_t)0x00020000U) /*!< 2 bytes data field */
4  #define FDCAN_DLC_BYTES_3 ((uint32_t)0x00030000U) /*!< 3 bytes data field */
5  #define FDCAN_DLC_BYTES_4 ((uint32_t)0x00040000U) /*!< 4 bytes data field */
6  #define FDCAN_DLC_BYTES_5 ((uint32_t)0x00050000U) /*!< 5 bytes data field */
7  #define FDCAN_DLC_BYTES_6 ((uint32_t)0x00060000U) /*!< 6 bytes data field */
8  #define FDCAN_DLC_BYTES_7 ((uint32_t)0x00070000U) /*!< 7 bytes data field */
9  #define FDCAN_DLC_BYTES_8 ((uint32_t)0x00080000U) /*!< 8 bytes data field */

```

```

10 #define FDCAN_DLC_BYTES_12 ((uint32_t)0x00090000U) /*!< 12 bytes data field */
11 #define FDCAN_DLC_BYTES_16 ((uint32_t)0x000A0000U) /*!< 16 bytes data field */
12 #define FDCAN_DLC_BYTES_20 ((uint32_t)0x000B0000U) /*!< 20 bytes data field */
13 #define FDCAN_DLC_BYTES_24 ((uint32_t)0x000C0000U) /*!< 24 bytes data field */
14 #define FDCAN_DLC_BYTES_32 ((uint32_t)0x000D0000U) /*!< 32 bytes data field */
15 #define FDCAN_DLC_BYTES_48 ((uint32_t)0x000E0000U) /*!< 48 bytes data field */
16 #define FDCAN_DLC_BYTES_64 ((uint32_t)0x000F0000U) /*!< 64 bytes data field */

```

- **ErrorStateIndicator**: 用于设置错误状态指示:

Code block

```

1 #define FDCAN_ESI_ACTIVE ((uint32_t)0x00000000U) /*!< 传输节点 error active */
2 #define FDCAN_ESI_PASSIVE ((uint32_t)0x80000000U) /*!< 传输节点 error passive */

```

- **BitRateSwitch**: 用于设置发送是否波特率可变。

Code block

```

1 #define FDCAN_BRS_OFF ((uint32_t)0x00000000U) /*!< FDCAN 帧发送/接收不带波特率可变 */
2 #define FDCAN_BRS_ON ((uint32_t)0x00100000U) /*!< FDCAN 帧发送/接收带波特率可变 */

```

- **FDFormat**: 用于设置发送帧是经典格式还是 CANFD 格式。

Code block

```

1 #define FDCAN_CLASSIC_CAN ((uint32_t)0x00000000U) /*!< 帧发送/接收使用经典 CAN */
2 #define FDCAN_FD_CAN ((uint32_t)0x00200000U) /*!< 帧发送/接收使用 FDCAN 格式 */

```

- **TxEvtFifoControl**: 用于设置发送事件 FIFO 控制。

Code block

```

1 #define FDCAN_NO_TX_EVENTS ((uint32_t)0x00000000U) /*!< 不存储 Tx events */
2 #define FDCAN_STORE_TX_EVENTS ((uint32_t)0x00800000U) /*!< 存储 Tx events */

```

- **MessageMarker**: 用于设置复制到 TX EVENT FIFO 的消息 Maker，来识别消息状态，范围 0 到 0xFF。

5.2 FDCAN 发送消息bsp代码

Code block

```
1  uint8_t fdcanx_send_data(hcan_t *hfdcan, uint16_t id, uint8_t *data, uint32_t
    len)
2  {
3      FDCAN_TxHeaderTypeDef pTxHeader;
4      pTxHeader.Identifier=id;           // 设置CAN ID
5      pTxHeader.IdType=FDCAN_STANDARD_ID; // 设置为标准ID
6      pTxHeader.TxFrameType=FDCAN_DATA_FRAME; // 设置为数据帧
7      // 设置发送的数据长度
8      if(len<=8)
9          pTxHeader.DataLength = len;
10     else if(len==12)
11         pTxHeader.DataLength = FDCAN_DLC_BYTES_12;
12     else if(len==16)
13         pTxHeader.DataLength = FDCAN_DLC_BYTES_16;
14     else if(len==20)
15         pTxHeader.DataLength = FDCAN_DLC_BYTES_20;
16     else if(len==24)
17         pTxHeader.DataLength = FDCAN_DLC_BYTES_24;
18     else if(len==32)
19         pTxHeader.DataLength = FDCAN_DLC_BYTES_32;
20     else if(len==48)
21         pTxHeader.DataLength = FDCAN_DLC_BYTES_48;
22     else if(len==64)
23         pTxHeader.DataLength = FDCAN_DLC_BYTES_64;
24
25     pTxHeader.ErrorStateIndicator=FDCAN_ESI_ACTIVE; // 传输节点 error active
26     pTxHeader.BitRateSwitch=FDCAN_BRS_ON;          // FDCAN 帧发送/接收带波特
    率可变
27     pTxHeader.FDFormat=FDCAN_FD_CAN;               // 设置为FDCAN帧格式，兼容
    CAN
28     pTxHeader.TxEventFifoControl=FDCAN_NO_TX_EVENTS; // 不存储 Tx events 事件
29     pTxHeader.MessageMarker=0;
30
31     if(HAL_FDCAN_AddMessageToTxFifoQ(hfdcan, &pTxHeader, data)!=HAL_OK)
32         return 1; //发送
33     return 0;
34 }
```

5.3 FDCAN 发送消息测试

1. 在main函数里面调用并下载代码

```
code block
1uint8_t send_data[8] = {0,1,2,3,4,5,6,7};
2int main(void)
3{
4    /* 截取添加的代码如下 */
5    bsp_fdcan_set_baud(&hfdcan1, CAN_CLASS, CAN_BR_1M);
6    can_filter_init();
7    HAL_FDCAN_Start(&hfdcan1);
8    while (1)
9    {
10        fdcanx_send_data(&hfdcan1, 0x520, send_data, 8);
11        HAL_Delay(500);
12    }
13 }
```

2. 接上USB转CAN，接线一定要正确，然后打开USB转CAN的上位机

选好串口号并设置好波特率，打开设备就可以看到发送的数据

USB2CAN

串口号:
COM3

波特率:
921600

数据位:
8

校验位:
None

停止位:
One

CAN		UART		CAN负载			
序号	时间戳	帧类型	帧格式	状态	数据长度	数据(D0 - D7)	ID
1	16:28:31.7063408	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
2	16:28:32.2093779	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
3	16:28:32.7097456	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
4	16:28:33.2109559	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
5	16:28:33.7135248	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
6	16:28:34.2128629	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
7	16:28:34.7124537	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
8	16:28:35.2134767	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
9	16:28:35.7153102	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
10	16:28:36.2156484	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
11	16:28:36.7172425	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
12	16:28:37.2186310	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
13	16:28:37.7207577	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
14	16:28:38.2213993	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
15	16:28:38.7229824	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
16	16:28:39.2226154	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
17	16:28:39.7230822	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
18	16:28:40.2242546	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520
19	16:28:40.7243665	标准帧	数据帧	接收	8	00 01 02 03 04 05 06 07	520

波特率(Kbps)
1000

帧ID(HEX)
000

发送数据(HEX)
D0 D1 D2 D3 D4 D5 D6 D7
12 23 34 45 56 67 78 89

发送次数
1

帧格式
标准帧

发送

定时发送
1 ms

帧类型
数据帧

停止

Connect

0 FPS

2 FPS

0.0%

发送帧数: 0

接收帧数: 19

6. FDCAN 接收数据

6.1 FDCAN 接收消息结构体

Code block

```
1  typedef struct
2  {
3      uint32_t Identifier;          /*!< Specifies the identifier.
4                                     This parameter must be a number between:
5                                     - 0 and 0x7FF, if IdType is
6                                     FDCAN_STANDARD_ID
7                                     - 0 and 0x1FFFFFFF, if IdType is
8                                     FDCAN_EXTENDED_ID
9                                     */
10     uint32_t IdType;              /*!< Specifies the identifier type for the
11                                     transmitted message.
12                                     This parameter can be a value of @ref
13                                     FDCAN_id_type
14                                     */
15     uint32_t TxFrameType;         /*!< Specifies the frame type of the
16                                     transmitted message.
17                                     This parameter can be a value of @ref
18                                     FDCAN_frame_type
19                                     */
20     uint32_t DataLength;          /*!< Specifies the length of the transmitted
21                                     frame.
22                                     This parameter can be a value of @ref
23                                     FDCAN_data_length_code
24                                     */
25     uint32_t ErrorStateIndicator; /*!< Specifies the error state indicator.
26                                     This parameter can be a value of @ref
27                                     FDCAN_error_state_indicator
28                                     */
29     uint32_t BitRateSwitch;       /*!< Specifies whether the Tx frame is
30                                     transmitted with or without bit
31                                     rate switching.
32                                     This parameter can be a value of @ref
33                                     FDCAN_bit_rate_switching
34                                     */
35     uint32_t FDFormat;            /*!< Specifies whether the Tx frame is
36                                     transmitted in classic or FD
37                                     format.
38                                     This parameter can be a value of @ref
39                                     FDCAN_format
40                                     */
41     uint32_t TxTimestamp;         /*!< Specifies the timestamp counter value
42                                     captured on start of frame
43                                     transmission.
```



```

30                                     This parameter must be a number between 0
      and 0xFFFF                      */
31
32     uint32_t MessageMarker;          /*!< Specifies the message marker copied into
      Tx Event FIFO element
33                                     for identification of Tx message status.
34                                     This parameter must be a number between 0
      and 0xFF                          */
35
36     uint32_t EventType;              /*!< Specifies the event type.
37                                     This parameter can be a value of @ref
      FDCAN_event_type                  */
38
39 } FDCAN_TxEventFifoTypeDef;

```

- **Identifier**: 用于设置 ID，如果 IdType 是标准 FDCAN_STANDARD_ID，范围 0 到 0x7FF，如果 IdType 是 FDCAN_EXTENDED_ID 扩展 ID，范围 0 到 0x1FFFFFFF。
- **IdType**: 用于设置标志 ID 或者扩展 ID。

Code block

```

1  #define FDCAN_STANDARD_ID ((uint32_t)0x00000000U) /*!< 标准 ID */
2  #define FDCAN_EXTENDED_ID ((uint32_t)0x40000000U) /*!< 扩展 ID */

```

- **RxFrameType**: 用于设置接收帧类型，数据帧或遥控帧

Code block

```

1  #define FDCAN_DATA_FRAME ((uint32_t)0x00000000U) /*!< 数据帧 */
2  #define FDCAN_REMOTE_FRAME ((uint32_t)0x20000000U) /*!< 远程帧 */

```

- **DataLength**: 用于设置数据长度。

Code block

```

1  #define FDCAN_DLC_BYTES_0 ((uint32_t)0x00000000U) /*!< 0 bytes data field */
2  #define FDCAN_DLC_BYTES_1 ((uint32_t)0x00010000U) /*!< 1 bytes data field */
3  #define FDCAN_DLC_BYTES_2 ((uint32_t)0x00020000U) /*!< 2 bytes data field */
4  #define FDCAN_DLC_BYTES_3 ((uint32_t)0x00030000U) /*!< 3 bytes data field */
5  #define FDCAN_DLC_BYTES_4 ((uint32_t)0x00040000U) /*!< 4 bytes data field */
6  #define FDCAN_DLC_BYTES_5 ((uint32_t)0x00050000U) /*!< 5 bytes data field */
7  #define FDCAN_DLC_BYTES_6 ((uint32_t)0x00060000U) /*!< 6 bytes data field */
8  #define FDCAN_DLC_BYTES_7 ((uint32_t)0x00070000U) /*!< 7 bytes data field */
9  #define FDCAN_DLC_BYTES_8 ((uint32_t)0x00080000U) /*!< 8 bytes data field */

```

```

10 #define FDCAN_DLC_BYTES_12 ((uint32_t)0x00090000U) /*!< 12 bytes data field */
11 #define FDCAN_DLC_BYTES_16 ((uint32_t)0x000A0000U) /*!< 16 bytes data field */
12 #define FDCAN_DLC_BYTES_20 ((uint32_t)0x000B0000U) /*!< 20 bytes data field */
13 #define FDCAN_DLC_BYTES_24 ((uint32_t)0x000C0000U) /*!< 24 bytes data field */
14 #define FDCAN_DLC_BYTES_32 ((uint32_t)0x000D0000U) /*!< 32 bytes data field */
15 #define FDCAN_DLC_BYTES_48 ((uint32_t)0x000E0000U) /*!< 48 bytes data field */
16 #define FDCAN_DLC_BYTES_64 ((uint32_t)0x000F0000U) /*!< 64 bytes data field */

```

- **ErrorStateIndicator**: 用于设置错误状态指示:

Code block

```

1 #define FDCAN_ESI_ACTIVE ((uint32_t)0x00000000U) /*!< 传输节点 error active */
2 #define FDCAN_ESI_PASSIVE ((uint32_t)0x80000000U) /*!< 传输节点 error passive */

```

- **BitRateSwitch**: 用于设置接收是否带波特率切换

Code block

```

1 #define FDCAN_BRS_OFF ((uint32_t)0x00000000U) /*!< FDCAN 帧发送/接收不支持波特率可
  变*/
2 #define FDCAN_BRS_ON ((uint32_t)0x00100000U) /*!< FDCAN 帧发送/接收支持波特率可变
  */

```

- **FDFormat**: 用于设置接收帧是经典格式还是 CANFD 格式

Code block

```

1 #define FDCAN_CLASSIC_CAN ((uint32_t)0x00000000U) /*!< 经典帧 */
2 #define FDCAN_FD_CAN ((uint32_t)0x00200000U) /*!< FDCAN 帧 */

```

- **RxTimestamp**: 用于设置帧接收时间戳，范围 0 到 0xFFFF。
- **FilterIndex**: 用于设置接收过滤索引。如果是标准 ID，范围 0 到 127，如果是扩展 ID，范围 0 到 63。
- **IsFilterMatchingFrame**: 用于设置是否接收非匹配帧，通过函数 HAL_FDCAN_ConfigGlobalFilter() 可以使能。0: 表示不接受; 1: 表示接收。

6.2 FDCAN 接收消息 bsp 代码

Code block

```

1  uint8_t fdcanx_receive(hcan_t *hfdcan, uint16_t *rec_id, uint8_t *buf)
2  {
3      FDCAN_RxHeaderTypeDef pRxHeader;
4      uint8_t len;
5
6      if(HAL_FDCAN_GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &pRxHeader,
buf)==HAL_OK)
7      {
8          *rec_id = pRxHeader.Identifier;
9          if(pRxHeader.DataLength<=FDCAN_DLC_BYTES_8)
10             len = pRxHeader.DataLength;
11          else if(pRxHeader.DataLength==FDCAN_DLC_BYTES_12)
12             len = 12;
13          else if(pRxHeader.DataLength==FDCAN_DLC_BYTES_16)
14             len = 16;
15          else if(pRxHeader.DataLength==FDCAN_DLC_BYTES_20)
16             len = 20;
17          else if(pRxHeader.DataLength==FDCAN_DLC_BYTES_24)
18             len = 24;
19          else if(pRxHeader.DataLength==FDCAN_DLC_BYTES_32)
20             len = 32;
21          else if(pRxHeader.DataLength==FDCAN_DLC_BYTES_48)
22             len = 48;
23          else if(pRxHeader.DataLength==FDCAN_DLC_BYTES_64)
24             len = 64;
25
26          return len; //接收数据的长度
27      }
28      return 0;
29  }

```

6.3 FDCAN 接收消息测试

1. 在main函数里面调用并下载代码

Code block

```

1  int main(void)
2  {
3      /* 截取添加的代码如下 */
4      bsp_fdcan_set_baud(&hfdcan1, CAN_CLASS, CAN_BR_1M); // 设置为经典can 波特率为
1M
5      can_filter_init();
6      HAL_FDCAN_Start(&hfdcan1);
7      HAL_FDCAN_ActivateNotification(&hfdcan1, FDCAN_IT_RX_FIFO0_WATERMARK); //
打开接收FIFO0中断

```

```

8     while (1)
9     {
10
11     }
12 }
13 uint8_t read_data[8] = {0};
14 uint16_t rec_id;
15 uint8_t len;
16 void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t
RxFifo0ITs)
17 {
18     if (hfdcan == &hfdcan1)
19     {
20         len=fdcanx_receive(&hfdcan1, &rec_id, read_data);
21     }
22 }
23

```

2. 接上USB转CAN，接线一定要正确，然后打开USB转CAN的上位机

3. 选好串口号并设置好波特率，然后发送数据，可以在keil的debug界面看到接收到的数据以及id和数据长度

The screenshot displays the USB2CAN software interface. On the left, the 'COM3' port is selected with a baud rate of '921600'. The main table shows four received CAN frames with ID 521 and length 8. The bottom section is for sending data, with ID 521 and data '12 23 34 45 56 67 78 89' entered. On the right, a variable watch window shows the contents of the 'read_data' array as ASCII characters: '0', '#', '4', 'E', 'V', 'g', 'x', '?'. Below this, 'rec_id' is 0x0521 and 'len' is 0x08.

序号	时间戳	帧类型	帧格式	状态	数据长度	数据(D0 - D7)	ID
1	17:06:57.7599721	标准帧	数据帧	发送成功	8	12 23 34 45 56 67 78 89	521
2	17:06:58.2089055	标准帧	数据帧	发送成功	8	12 23 34 45 56 67 78 89	521
3	17:06:59.1335684	标准帧	数据帧	发送成功	8	12 23 34 45 56 67 78 89	521
4	17:07:00.1683342	标准帧	数据帧	发送成功	8	12 23 34 45 56 67 78 89	521

Variable	Value	Type
read_data	0x24000018 read_data[...]	uchar[8]
[0]	0x12	uchar
[1]	0x23 '#'	uchar
[2]	0x34 '4'	uchar
[3]	0x45 'E'	uchar
[4]	0x56 'V'	uchar
[5]	0x67 'g'	uchar
[6]	0x78 'x'	uchar
[7]	0x89 '?'	uchar
rec_id	0x0521	ushort
len	0x08	uchar

7. 电机底层控制API

7.1 反馈帧

1. 反馈报文格式说明：

反馈ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
MST_ID D	ID ERR<< 4	POS[15: 8]	POS[7:0]	VEL[11: 4]	VEL[3:0] T[11: 8]	T[7:0]	T_MOS	T_Rotor

- MST_ID：电机内部设置的 **master id** 默认为0
- ID：表示控制器的 ID，取 CAN_ID 的低 8 位

⚠ 注意：因为这里D[0]是CAN ID与状态位融合在一起的，所以电机内部 CAN ID 的设置不能大于 15，不然会溢出到状态位，导致电机莫名其妙报错的情况

- ERR：表示状态，对应状态类型为：
 - 0——失能；
 - 1——使能；
 - 2——输出轴编码器未校准；
 - 8——超压；
 - 9——欠压；
 - A——过电流；
 - B——MOS 过温；
 - C——电机线圈过温；
 - D——通讯丢失；
 - E——过载；
- POS：表示电机的位置信息
- VEL：表示电机的速度信息
- T：表示电机的扭矩信息
- T_MOS：表示驱动上 MOS 的平均温度，单位℃
- T_Rotor：表示电机内部线圈的平均温度，单位℃

⚠ 注意：位置、速度和扭矩采用线性映射的关系将浮点型数据转换成有符号的定点数据，其中位置采用 16 位数据，速度和扭矩均使用 12 位。

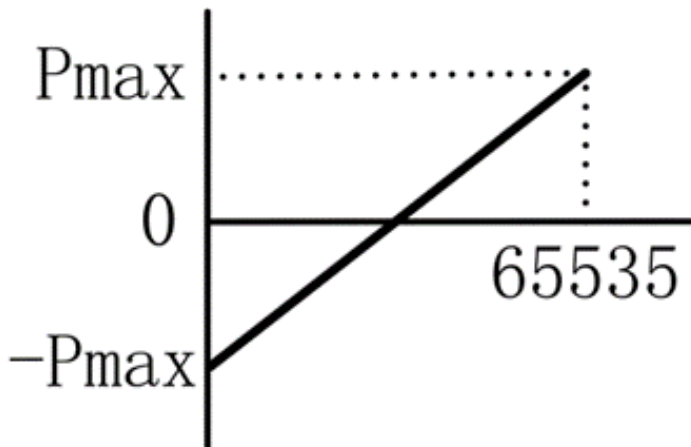
2. 反馈报文解析API，按上面报文格式进行解析

Code block

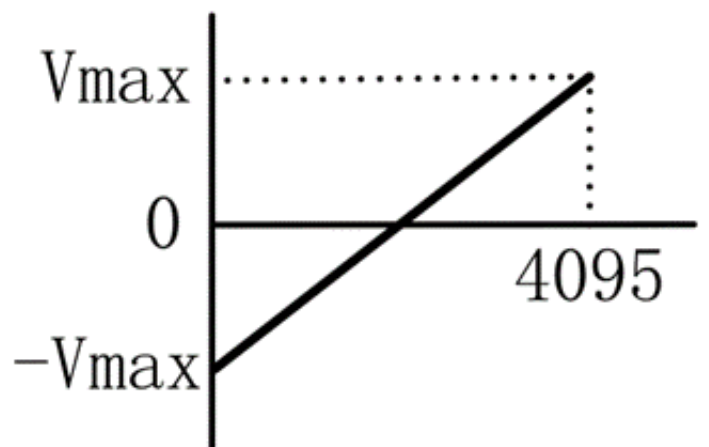
```

1  void dm_motor_fbdata(motor_t *motor, uint8_t *rx_data)
2  {
3      motor->para.id = (rx_data[0])&0x0F;
4      motor->para.state = (rx_data[0])>>4;
5      motor->para.p_int=(rx_data[1]<<8)|rx_data[2];
6      motor->para.v_int=(rx_data[3]<<4)|(rx_data[4]>>4);
7      motor->para.t_int=((rx_data[4]&0xF)<<8)|rx_data[5];
8      motor->para.pos = uint_to_float(motor->para.p_int, -motor->tmp.PMAX, motor-
>tmp.PMAX, 16); // (-12.5,12.5)
9      motor->para.vel = uint_to_float(motor->para.v_int, -motor->tmp.VMAX, motor-
>tmp.VMAX, 12); // (-45.0,45.0)
10     motor->para.tor = uint_to_float(motor->para.t_int, -motor->tmp.TMAX, motor-
>tmp.TMAX, 12); // (-18.0,18.0)
11     motor->para.Tmos = (float)(rx_data[6]);
12     motor->para.Tcoil = (float)(rx_data[7]);
13 }

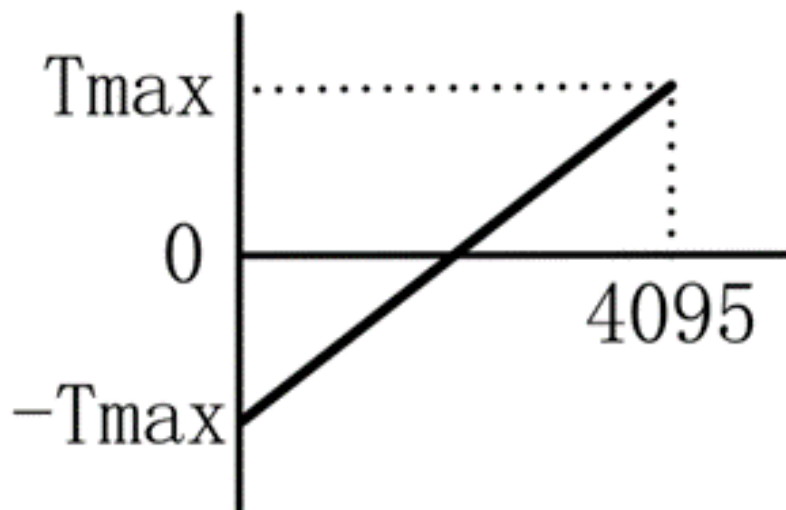
```



位置线性映射图



速度线性映射图



扭矩线性映射图

⚠ 注意：上面代码解析中的 P_{MAX}、V_{MAX}、T_{MAX} 需要与电机内部设置的一致

7.2 MIT 模式控制帧

1. 控制报文格式说明：

控制ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
ID	p_des[15:8]	p_des[7:0]	v_des[11:4]	v_des[3:0]] Kp[11:8]	Kp[7:0]	Kd[11:4]	Kd[3:0] t_ff[11:8]	t_ff[7:0]

- 控制帧ID：等于设定的 CAN ID 值
- P_{des}：位置给定 单位rad
- V_{des}：速度给定 单位rad/s
- K_p：位置比例系数
- K_d：位置微分系数
- T_{ff}：转矩给定值 单位Nm

标准 CAN 数据一帧只有 8 个字节，MIT 的控制命令格式将 Position、Velocity、K_p、K_d、Torque 五个参数按位组合在 8 个字节中。其中：Position 占用 2 个字节 16 位、Velocity 占用 12 位、K_p 占用 12 位、K_d 占用 12 位。K_p 的范围为[0,500]，K_d 的范围为[0,5]。

2. 控制报文解析API

- MIT 命令采用浮点数据等比例转换成整数发送到驱动器，驱动器再将接收到的整数等比例转换成浮点数据。这部分转换使用到 `uint_to_float`、`float_to_uint` 两个转换函数进行转换，该部分具体实现如下所示：

Code block

```
1  int float_to_uint(float x_float, float x_min, float x_max, int bits)
2  {
3      /* Converts a float to an unsigned int, given range and number of bits */
4      float span = x_max - x_min;
5      float offset = x_min;
6      return (int) ((x_float-offset)*((float)((1<<bits)-1))/span);
7  }
8
9  float uint_to_float(int x_int, float x_min, float x_max, int bits)
10 {
```

```

11      /* converts unsigned int to float, given range and number of bits */
12      float span = x_max - x_min;
13      float offset = x_min;
14      return ((float)x_int)*span/((float)((1<<bits)-1)) + offset;
15  }

```

⚠ 注意：这两个转换函数需要首先确定两个等比例转换的最大最小值，这两个值可以在参数设定页面查询，其中 KP、KD 的最大最小值默认分别为 0.0~500.0、0.0~5.0。Pos、Vel、Torque 如下图所示分别预设为 ±12.5、±45、±18，这三个参数可以根据电机的实际参数进行调整。但发送控制命令时，一定要与设定值保持一致，否则会控制命令会发生等比例缩放。

- MIT控制模式API

Code block

```

1  void mit_ctrl(hcan_t* hcan, motor_t *motor, uint16_t motor_id, float pos, float
    vel, float kp, float kd, float tor)
2  {
3      uint8_t data[8];
4      uint16_t pos_tmp, vel_tmp, kp_tmp, kd_tmp, tor_tmp;
5      uint16_t id = motor_id + MIT_MODE;
6
7      pos_tmp = float_to_uint(pos, -motor->tmp.PMAX, motor->tmp.PMAX, 16);
8      vel_tmp = float_to_uint(vel, -motor->tmp.VMAX, motor->tmp.VMAX, 12);
9      tor_tmp = float_to_uint(tor, -motor->tmp.TMAX, motor->tmp.TMAX, 12);
10     kp_tmp = float_to_uint(kp, KP_MIN, KP_MAX, 12);
11     kd_tmp = float_to_uint(kd, KD_MIN, KD_MAX, 12);
12
13     data[0] = (pos_tmp >> 8);
14     data[1] = pos_tmp;
15     data[2] = (vel_tmp >> 4);
16     data[3] = ((vel_tmp & 0xF) << 4) | (kp_tmp >> 8);
17     data[4] = kp_tmp;
18     data[5] = (kd_tmp >> 4);
19     data[6] = ((kd_tmp & 0xF) << 4) | (tor_tmp >> 8);
20     data[7] = tor_tmp;
21
22     fdcanx_send_data(hcan, id, data, 8);
23 }

```

7.3 位置速度模式控制帧

1. 控制报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
0x100+ID	p_des				v_des			

- 报文ID：等于设定的 CAN ID 值 + 0x100
- P_des：位置给定，浮点型，低位在前，高位在后 单位rad
- V_des：速度给定，浮点型，低位在前，高位在后 单位rad/s

2. 控制报文API

Code block

```
1 void pos_ctrl(hcan_t* hcan, uint16_t motor_id, float pos, float vel)
2 {
3     uint16_t id;
4     uint8_t *pbuf, *vbuf;
5     uint8_t data[8];
6
7     id = motor_id + POS_MODE;
8     pbuf = (uint8_t*)&pos;
9     vbuf = (uint8_t*)&vel;
10
11     data[0] = *pbuf;
12     data[1] = *(pbuf+1);
13     data[2] = *(pbuf+2);
14     data[3] = *(pbuf+3);
15
16     data[4] = *vbuf;
17     data[5] = *(vbuf+1);
18     data[6] = *(vbuf+2);
19     data[7] = *(vbuf+3);
20
21     fdcanx_send_data(hcan, id, data, 8);
22 }
```

⚠ 注意：API函数内部已经加了 0x100 的偏移

7.4 速度模式控制帧

1. 控制报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]
0x200+ID	v_des			

- 报文ID： 等于设定的 CAN ID 值 + 0x200
- V_des： 速度给定，浮点型，低位在前，高位在后 单位rad/s

2. 控制报文API

Code block

```
1 void spd_ctrl(hcan_t* hcan, uint16_t motor_id, float vel)
2 {
3     uint16_t id;
4     uint8_t *vbuf;
5     uint8_t data[4];
6
7     id = motor_id + SPD_MODE;
8     vbuf=(uint8_t*)&vel;
9
10    data[0] = *vbuf;
11    data[1] = *(vbuf+1);
12    data[2] = *(vbuf+2);
13    data[3] = *(vbuf+3);
14
15    fdcanx_send_data(hcan, id, data, 4);
16 }
```

⚠ 注意：API函数内部已经加了 0x200 的偏移

7.5 力位混控模式控制帧

1. 控制报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
0x300+ID	p_des				v_des		i_des	

- 报文ID： 等于设定的 CAN ID 值 + 0x300
- p_des： 位置给定，单位为rad，浮点类型

- v_des: 限速值, 单位rad/s, 放大**100倍**, 类型为无符号16位整型, 低位在前, 高位在后, 范围为0-10000, 超过10000会限制在10000, 故对应的实际速度限定幅值为0~100rad/s
- i_des: 扭矩电流限定标么值, 放大**10000倍**, 类型为无符号16位, 低位在前, 高位在后, 范围为0-10000, 超过10000会限制在10000, 对应的实际电流限定标么幅值为0-1.0
- 电流标么值: 实际电流值除以最大电流值, 最大电流见电机上电串口打印的 **I_{max}**。

2. 控制报文API

Code block

```
1 void psi_ctrl(hcan_t* hcan, uint16_t motor_id, float pos, float vel, float cur)
2 {
3     uint16_t id;
4     uint8_t *pbuf, *vbuf, *ibuf;
5     uint8_t data[8];
6
7     uint16_t u16_vel = vel*100;
8     uint16_t u16_cur = cur*10000;
9
10    id = motor_id + PSI_MODE;
11    pbuf=(uint8_t*)&pos;
12    vbuf=(uint8_t*)&u16_vel;
13    ibuf=(uint8_t*)&u16_cur;
14
15    data[0] = *pbuf;
16    data[1] = *(pbuf+1);
17    data[2] = *(pbuf+2);
18    data[3] = *(pbuf+3);
19
20    data[4] = *vbuf;
21    data[5] = *(vbuf+1);
22
23    data[6] = *ibuf;
24    data[7] = *(ibuf+1);
25
26    fdcanx_send_data(hcan, id, data, 8);
27 }
```

⚠ 注意: API函数内部已经加了 **0x300** 的偏移, 并将浮点类型的速度和电流设置值转化为了整型

7.6 使能帧

1. 控制报文格式说明:

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
mode_id+ID	FF	FF	FF	FF	FF	FF	FF	FC

- 报文ID：等于电机设定的 CAN ID 值 + 模式偏移ID
- CAN ID：电机设置的CAN ID
- 模式偏移ID：MIT模式偏移0x00，位置速度模式偏移0x100，速度模式偏移0x200，力位混控模式偏移0x300

2. 控制报文API

Code block

```
1 void enable_motor_mode(hcan_t* hcan, uint16_t motor_id, uint16_t mode_id)
2 {
3     uint8_t data[8];
4     uint16_t id = motor_id + mode_id;
5
6     data[0] = 0xFF;
7     data[1] = 0xFF;
8     data[2] = 0xFF;
9     data[3] = 0xFF;
10    data[4] = 0xFF;
11    data[5] = 0xFF;
12    data[6] = 0xFF;
13    data[7] = 0xFC;
14
15    fdcanx_send_data(hcan, id, data, 8);
16 }
```

⚠ 注意：motor_id就是电机内部设置的 CAN ID

7.7 失能帧

1. 控制报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
mode_id+ID	FF	FF	FF	FF	FF	FF	FF	FD

- 报文ID：等于电机设定的 CAN ID 值 + 模式偏移ID

- CAN ID：电机设置的CAN ID
- 模式偏移ID：MIT模式偏移0x00，位置速度模式偏移0x100，速度模式偏移0x200，力位混控模式偏移0x300

2. 控制报文API

Code block

```
1 void disable_motor_mode(hcan_t* hcan, uint16_t motor_id, uint16_t mode_id)
2 {
3     uint8_t data[8];
4     uint16_t id = motor_id + mode_id;
5
6     data[0] = 0xFF;
7     data[1] = 0xFF;
8     data[2] = 0xFF;
9     data[3] = 0xFF;
10    data[4] = 0xFF;
11    data[5] = 0xFF;
12    data[6] = 0xFF;
13    data[7] = 0xFD;
14
15    fdcanx_send_data(hcan, id, data, 8);
16 }
```

⚠ 注意：motor_id 就是电机内部设置的 CAN ID

7.8 设置零点

1. 控制报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
mode_id+ID	FF	FF	FF	FF	FF	FF	FF	FE

- 报文ID：等于电机设定的 CAN ID 值 + 模式偏移ID
- CAN ID：电机设置的CAN ID
- 模式偏移ID：MIT模式偏移0x00，位置速度模式偏移0x100，速度模式偏移0x200，力位混控模式偏移0x300

2. 控制报文API

```

1 void save_pos_zero(hcan_t* hcan, uint16_t motor_id, uint16_t mode_id)
2 {
3     uint8_t data[8];
4     uint16_t id = motor_id + mode_id;
5
6     data[0] = 0xFF;
7     data[1] = 0xFF;
8     data[2] = 0xFF;
9     data[3] = 0xFF;
10    data[4] = 0xFF;
11    data[5] = 0xFF;
12    data[6] = 0xFF;
13    data[7] = 0xFE;
14
15    fdcanx_send_data(hcan, id, data, 8);
16 }

```

⚠ 注意：motor_id 就是电机内部设置的 CAN ID

7.9 清除错误

1. 控制报文格式说明：

控制ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
mode_id+ID	FF	FF	FF	FF	FF	FF	FF	FB

- 控制ID：等于电机设定的 CAN ID 值 + 模式偏移ID
- CAN ID：电机设置的CAN ID
- 模式偏移ID：MIT模式偏移0x00，位置速度模式偏移0x100，速度模式偏移0x200，力位混控模式偏移0x300

2. 控制报文API

Code block

```

1 void clear_err(hcan_t* hcan, uint16_t motor_id, uint16_t mode_id)
2 {
3     uint8_t data[8];
4     uint16_t id = motor_id + mode_id;
5
6     data[0] = 0xFF;
7     data[1] = 0xFF;

```

```

8         data[2] = 0xFF;
9         data[3] = 0xFF;
10        data[4] = 0xFF;
11        data[5] = 0xFF;
12        data[6] = 0xFF;
13        data[7] = 0xFB;
14
15        fdcanx_send_data(hcan, id, data, 8);
16    }

```

⚠ 注意：motor_id 就是电机内部设置的 CAN ID

7.10 封装API函数

1. 电机结构体说明

- 电机的反馈信息结构体

Code block

```

1  typedef struct
2  {
3      int id;                // 电机内部设置的 can id
4      int state;            // 电机状态
5      int p_int;            // 整型位置信息
6      int v_int;            // 整型速度信息
7      int t_int;            // 整型扭矩信息
8      int kp_int;           // 整型Kp信息
9      int kd_int;           // 整型Kd信息
10     float pos;             // 最终解析出来的位置信息
11     float vel;             // 最终解析出来的速度信息
12     float tor;             // 最终解析出来的扭矩信息
13     float Kp;              // 最终解析出来的Kp数据
14     float Kd;              // 最终解析出来的Kd数据
15     float Tmos;            // 板子MOS温度
16     float Tcoil;          // 电机线圈温度
17 } motor_fbpara_t;

```

- 电机的控制参数结构体

Code block

```

1  typedef struct
2  {
3      uint8_t mode;         // 电机控制模式

```

```

4     float pos_set;           // 电机位置设定
5     float vel_set;           // 电机速度设定
6     float tor_set;           // 电机扭矩设定
7     float cur_set;           // 电机电流设定
8     float kp_set;            // Kp设定
9     float kd_set;            // Kd设定
10 } motor_ctrl_t;

```

- ⚠ 1. MIT模式使用的参数：pos_set, vel_set, tor_set, kp_set, kd_set
2. 位置速度模式使用的参数：pos_set, vel_set
3. 速度模式使用的参数：vel_set
4. 力位混孔模式使用的参数：pos_set, vel_set, cur_set

- 电机主结构体

Code block

```

1  typedef struct
2  {
3      uint16_t id;           // can id
4      uint16_t mst_id;       // master id 标识作用
5      motor_fbpara_t para;   // 电机的反馈信息结构体
6      motor_ctrl_t ctrl;     // 电机的控制参数结构体
7      esc_inf_t tmp;         // 电机的寄存器参数结构体
8  } motor_t;
9

```

2. 封装使能API函数

Code block

```

1  // 通过判断 motor 结构体中的控制模式来选择使能的模式，并添加偏移的ID
2  void dm_motor_enable(hcan_t* hcan, motor_t *motor)
3  {
4      switch(motor->ctrl.mode)
5      {
6          case mit_mode:
7              enable_motor_mode(hcan, motor->id, MIT_MODE);
8              break;
9          case pos_mode:
10             enable_motor_mode(hcan, motor->id, POS_MODE);
11             break;
12          case spd_mode:

```



```

13         enable_motor_mode(hcan, motor->id, SPD_MODE);
14         break;
15     case psi_mode:
16         enable_motor_mode(hcan, motor->id, PSI_MODE);
17         break;
18     }
19 }

```

3. 封装失能API函数

Code block

```

1  // 通过判断 motor 结构体中的控制模式来选择失能的模式，并添加偏移的ID
2  void dm_motor_disable(hcan_t* hcan, motor_t *motor)
3  {
4      switch(motor->ctrl.mode)
5      {
6          case mit_mode:
7              disable_motor_mode(hcan, motor->id, MIT_MODE);
8              break;
9          case pos_mode:
10             disable_motor_mode(hcan, motor->id, POS_MODE);
11             break;
12          case spd_mode:
13             disable_motor_mode(hcan, motor->id, SPD_MODE);
14             break;
15          case psi_mode:
16             disable_motor_mode(hcan, motor->id, PSI_MODE);
17             break;
18      }
19      dm_motor_clear_para(motor);
20 }

```

4. 封装清除错误函数

Code block

```

1  // 通过判断 motor 结构体中的控制模式来选择清除错误的模式，并添加偏移的ID
2  void dm_motor_clear_err(hcan_t* hcan, motor_t *motor)
3  {
4      switch(motor->ctrl.mode)
5      {
6          case mit_mode:
7              clear_err(hcan, motor->id, MIT_MODE);
8              break;
9          case pos_mode:

```

```

10         clear_err(hcan, motor->id, POS_MODE);
11         break;
12     case spd_mode:
13         clear_err(hcan, motor->id, SPD_MODE);
14         break;
15     case psi_mode:
16         clear_err(hcan, motor->id, PSI_MODE);
17         break;
18     }
19 }

```

5. 封装模式控制函数

Code block

```

1  void dm_motor_ctrl_send(hcan_t* hcan, motor_t *motor)
2  {
3      switch(motor->ctrl.mode)
4      {
5          case mit_mode:
6              mit_ctrl(hcan, motor, motor->id, motor->ctrl.pos_set, motor-
>ctrl.vel_set, motor->ctrl.kp_set, motor->ctrl.kd_set, motor->ctrl.tor_set);
7              break;
8          case pos_mode:
9              pos_ctrl(hcan, motor->id, motor->ctrl.pos_set, motor-
>ctrl.vel_set);
10             break;
11          case spd_mode:
12              spd_ctrl(hcan, motor->id, motor->ctrl.vel_set);
13              break;
14          case psi_mode:
15              psi_ctrl(hcan, motor->id, motor->ctrl.pos_set, motor->ctrl.vel_set,
motor->ctrl.cur_set);
16              break;
17      }
18  }

```

8. 电机寄存器读写API

8.1 寄存器列表

地址(HEX)	地址(DEC)	变量	描述	读写	范围	类型

0x00	0	UV_Value	低压保护值	RW	(10.0,fmax]	float
0x01	1	KT_Value	扭矩系数	RW	[0.0,fmax]	float
0x02	2	OT_Value	过温保护值	RW	[80.0,200)	float
0x03	3	OC_Value	过流保护值	RW	(0.0,1.0)	float
0x04	4	ACC	加速度	RW	(0.0,fmax)	float
0x05	5	DEC	减速度	RW	[-fmax,0.0)	float
0x06	6	MAX_SPD	最大速度	RW	(0.0,fmax]	float
0x07	7	MST_ID	反馈ID	RW	[0,0x7FF]	uint32
0x08	8	ESC_ID	接收ID	RW	[0,0x7FF]	uint32
0x09	9	TIMEOUT	超时警报时间	RW	[0,2^32-1]	uint32
0x0A	10	CTRL_MODE	控制模式	RW	[0,4]	uint32
0x0B	11	Damp	电机粘滞系数	RO	/	float
0x0C	12	Inertia	电机转动惯量	RO	/	float
0x0D	13	hw_ver	保留	RO	/	uint32
0x0E	14	sw_ver	软件版本号	RO	/	uint32
0x0F	15	SN	保留	RO	/	uint32
0x10	16	NPP	电机极对数	RO	/	uint32
0x11	17	Rs	电机相电阻	RO	/	float
0x12	18	Ls	电机相电感	RO	/	float
0x13	19	Flux	电机磁链值	RO	/	float
0x14	20	Gr	齿轮减速比	RO	/	float
0x15	21	PMAX	位置映射范围	RW	(0.0,fmax]	float
0x16	22	VMAX	速度映射范围	RW	(0.0,fmax]	float
0x17	23	TMAX	扭矩映射范围	RW	(0.0,fmax]	float
0x18	24	I_BW	电流环控制带宽	RW	[100.0,1.0e4]	float
0x19	25	KP_ASR	速度环Kp	RW	[0.0,fmax]	float

0x1A	26	KI_ASR	速度环Ki	RW	[0.0,fmax]	float
0x1B	27	KP_APR	位置环Kp	RW	[0.0,fmax]	float
0x1C	28	KI_APR	位置环Ki	RW	[0.0,fmax]	float
0x1D	29	OV_Value	过压保护值	RW	TBD	float
0x1E	30	GREF	齿轮力矩效率	RW	(0.0,1.0]	float
0x1F	31	Deta	速度环阻尼系数	RW	[1.0,30.0]	float
0x20	32	V_BW	速度环滤波带宽	RW	(0.0,500.0)	float
0x21	33	IQ_c1	电流环增强系数	RW	[100.0, 1.0e4]	float
0x22	34	VL_c1	速度环增强系数	RW	(0.0,1.0e4]	float
0x23	35	can_br	CAN波特率	RW	[0,9]	uint32
0x24	36	sub_ver	子版本号	RO	/	uint32
0x32	50	u_off	u相偏置	RO	/	float
0x33	51	v_off	v相偏置	RO	/	float
0x34	52	k1	补偿因子1	RO	/	float
0x35	53	k2	补偿因子2	RO	/	float
0x36	54	m_off	角度偏移	RO	/	float
0x37	55	dir	方向	RO	/	float
0x50	80	p_m	电机当前位置	RO	/	float
0x51	81	xout	输出轴位置	RO	/	float

用户主要需要修改的参数：

地址(HEX)	地址(DEC)	变量	描述	读写	范围	类型
0x0A	10	CTRL_MODE	控制模式	RW	[0,4]	uint32
0x23	35	can_br	CAN波特率	RW	[0,9]	uint32

- 控制模式：

编码	模式
1	MIT
2	位置速度
3	速度
4	力位混控

- CAN 波特率：

编码	模式
0	125K
1	200K
2	250K
3	500K
4	1M
5	2M
6	2.5M
7	3.2M
8	4M
9	5M

8.2 读取寄存器参数

- 发送报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
0x7FF	CANID_L	CANID_H	0x33	RID	xx			

- 报文ID：0x7FF为广播ID，无论是那个电机都可以接收到，电机可以通过判断发送帧中的 CANID 来确定是哪一个电机需要读取或者修改寄存器传输

- CANID_L: CAN ID的低8位
- CANID_H: CAN ID的高8位，因为我们使用的是标准帧，所以该位一般为0
- 0x33: 代表读取寄存器指令
- RID: 表示读取的电机寄存器
- xx: 不需要关心这部分数据的多少，可以发长度为4的读取指令，也可以发长度为8的，这部分可要可不要。

2. 反馈报文格式说明

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
MST_ID	CANID_L	CANID_H	0x33	RID	data			

- MST_ID: 电机内部设置的 **master id** 默认为0
- CANID_L: CAN ID的低8位
- CANID_H: CAN ID的高8位，因为我们使用的是标准帧，所以该位一般为0
- 0x33: 代表读取寄存器指令
- RID: 表示要读取的电机寄存器
- data: 反馈读取电机寄存器的参数，低位在前，高位在后

3. 控制报文API

Code block

```
1 // 发送读取电机寄存器命令
2 void read_motor_data(uint16_t id, uint8_t rid)
3 {
4     uint8_t can_id_l = id & 0x0F;
5     uint8_t can_id_h = (id >> 4) & 0x0F;
6     uint8_t data[4] = {can_id_l, can_id_h, 0x33, rid};
7     fdcanx_send_data(&hfdcan1, 0x7FF, data, 4);
8 }
9 // 对读取电机反馈的命令进行解析
10 void receive_motor_data(motor_t *motor, uint8_t *data)
11 {
12     if(motor->tmp.read_flag == 0)
13         return ;
14
15     float_type_u y;
16
17     if(data[2] == 0x33)
```

```

18     {
19         uint16_t rid_value = data[3];
20         y.b_val[0] = data[4];
21         y.b_val[1] = data[5];
22         y.b_val[2] = data[6];
23         y.b_val[3] = data[7];
24
25         switch (rid_value)
26         {
27             case RID_UV_VALUE: motor->tmp.UV_Value = y.f_val; motor-
>tmp.read_flag = 2; break;
28             case RID_KT_VALUE: motor->tmp.KT_Value = y.f_val; motor-
>tmp.read_flag = 3; break;
29             case RID_OT_VALUE: motor->tmp.OT_Value = y.f_val; motor-
>tmp.read_flag = 4; break;
30             case RID_OC_VALUE: motor->tmp.OC_Value = y.f_val; motor-
>tmp.read_flag = 5; break;
31             case RID_ACC:      motor->tmp.ACC      = y.f_val; motor-
>tmp.read_flag = 6; break;
32             case RID_DEC:      motor->tmp.DEC      = y.f_val; motor-
>tmp.read_flag = 7; break;
33             case RID_MAX_SPD:  motor->tmp.MAX_SPD   = y.f_val; motor-
>tmp.read_flag = 8; break;
34             case RID_MST_ID:   motor->tmp.MST_ID    = y.u_val; motor-
>tmp.read_flag = 9; break;
35             case RID_ESC_ID:   motor->tmp.ESC_ID    = y.u_val; motor-
>tmp.read_flag = 10; break;
36             case RID_TIMEOUT:  motor->tmp.TIMEOUT   = y.u_val; motor-
>tmp.read_flag = 11; break;
37             case RID_CMODE:    motor->tmp.cmode     = y.u_val; motor-
>tmp.read_flag = 12; break;
38             case RID_DAMP:     motor->tmp.Damp      = y.f_val; motor-
>tmp.read_flag = 13; break;
39             case RID_INERTIA:  motor->tmp.Inertia   = y.f_val; motor-
>tmp.read_flag = 14; break;
40             case RID_HW_VER:   motor->tmp.hw_ver    = y.u_val; motor-
>tmp.read_flag = 15; break;
41             case RID_SW_VER:   motor->tmp.sw_ver    = y.u_val; motor-
>tmp.read_flag = 16; break;
42             case RID_SN:       motor->tmp.SN        = y.u_val; motor-
>tmp.read_flag = 17; break;
43             case RID_NPP:      motor->tmp.NPP       = y.u_val; motor-
>tmp.read_flag = 18; break;
44             case RID_RS:       motor->tmp.Rs        = y.f_val; motor-
>tmp.read_flag = 19; break;
45             case RID_LS:       motor->tmp.Ls        = y.f_val; motor-
>tmp.read_flag = 20; break;

```

```

46         case RID_FLUX:      motor->tmp.Flux      = y.f_val; motor-
>tmp.read_flag = 21; break;
47         case RID_GR:       motor->tmp.Gr        = y.f_val; motor-
>tmp.read_flag = 22; break;
48         case RID_PMAX:     motor->tmp.PMAX       = y.f_val; motor-
>tmp.read_flag = 23; break;
49         case RID_VMAX:     motor->tmp.VMAX       = y.f_val; motor-
>tmp.read_flag = 24; break;
50         case RID_TMAX:     motor->tmp.TMAX       = y.f_val; motor-
>tmp.read_flag = 25; break;
51         case RID_I_BW:     motor->tmp.I_BW       = y.f_val; motor-
>tmp.read_flag = 26; break;
52         case RID_KP_ASR:   motor->tmp.KP_ASR     = y.f_val; motor-
>tmp.read_flag = 27; break;
53         case RID_KI_ASR:   motor->tmp.KI_ASR     = y.f_val; motor-
>tmp.read_flag = 28; break;
54         case RID_KP_APR:   motor->tmp.KP_APR     = y.f_val; motor-
>tmp.read_flag = 29; break;
55         case RID_KI_APR:   motor->tmp.KI_APR     = y.f_val; motor-
>tmp.read_flag = 30; break;
56         case RID_OV_VALUE: motor->tmp.OV_Value   = y.f_val; motor-
>tmp.read_flag = 31; break;
57         case RID_GREF:     motor->tmp.GREF       = y.f_val; motor-
>tmp.read_flag = 32; break;
58         case RID_DETA:     motor->tmp.Deta       = y.f_val; motor-
>tmp.read_flag = 33; break;
59         case RID_V_BW:     motor->tmp.V_BW       = y.f_val; motor-
>tmp.read_flag = 34; break;
60         case RID_IQ_CL:    motor->tmp.IQ_cl      = y.f_val; motor-
>tmp.read_flag = 35; break;
61         case RID_VL_CL:    motor->tmp.VL_cl      = y.f_val; motor-
>tmp.read_flag = 36; break;
62         case RID_CAN_BR:   motor->tmp.can_br     = y.u_val; motor-
>tmp.read_flag = 37; break;
63         case RID_SUB_VER:  motor->tmp.sub_ver    = y.u_val; motor-
>tmp.read_flag = 38; break;
64         case RID_U_OFF:    motor->tmp.u_off      = y.f_val; motor-
>tmp.read_flag = 39; break;
65         case RID_V_OFF:    motor->tmp.v_off      = y.f_val; motor-
>tmp.read_flag = 40; break;
66         case RID_K1:       motor->tmp.k1        = y.f_val; motor-
>tmp.read_flag = 41; break;
67         case RID_K2:       motor->tmp.k2        = y.f_val; motor-
>tmp.read_flag = 42; break;
68         case RID_M_OFF:    motor->tmp.m_off      = y.f_val; motor-
>tmp.read_flag = 43; break;

```



```

69         case RID_DIR:      motor->tmp.dir      = y.f_val; motor-
    >tmp.read_flag = 44; break;
70         case RID_P_M:      motor->tmp.p_m      = y.f_val; motor-
    >tmp.read_flag = 45; break;
71         case RID_X_OUT:    motor->tmp.x_out     = y.f_val; motor-
    >tmp.read_flag = 0 ; break;
72     }
73 }
74 }

```

8.3 写入寄存器参数

1. 发送报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
0x7FF	CANID_L	CANID_H	0x55	RID	data			

- 报文ID：0x7FF为广播ID，无论是那个电机都可以接收到，电机可以通过判断发送帧中的 CANID 来确定是哪一个电机需要读取或者修改寄存器传输
- CANID_L：CAN ID的低8位
- CANID_H：CAN ID的高8位，因为我们使用的是标准帧，所以该位一般为0
- 0x55：代表写入寄存器指令
- RID：表示要写入的电机寄存器
- data：表示需要写入寄存器的数据，低位在前，高位在后

2. 反馈报文格式说明

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
MST_ID	CANID_L	CANID_H	0x55	RID	data			

- MST_ID：电机内部设置的 **master id** 默认为0
- CANID_L：CAN ID的低8位
- CANID_H：CAN ID的高8位，因为我们使用的是标准帧，所以该位一般为0
- 0x33：代表读取寄存器指令
- RID：表示电机寄存器地址

- data：反馈该电机寄存器的参数，低位在前，高位在后

⚠ 注意：写寄存器数据的命令会立即生效，但无法进行存储，掉电后会丢失，需要发送存储参数的指令

3. 控制报文API

Code block

```
1 // 发送写入电机寄存器命令
2 void write_motor_data(uint16_t id, uint8_t rid, uint8_t d0, uint8_t d1, uint8_t d2, uint8_t d3)
3 {
4     uint8_t can_id_l = id & 0xFF; // 低 8 位
5     uint8_t can_id_h = (id >> 8) & 0x07; // 高 3 位
6     uint8_t data[8] = {can_id_l, can_id_h, 0x55, rid, d0, d1, d2, d3};
7     fdcanx_send_data(&hfdcan1, 0x7FF, data, 8);
8 }
```

8.4 保存寄存器参数

1. 发送报文格式说明：

报文ID	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
0x7FF	CANID_L	CANID_H	0xAA	RID	xx			

- 报文ID：0x7FF为广播ID，无论是那个电机都可以接收到，电机可以通过判断发送帧中的 CANID 来确定是哪一个电机需要读取或者修改寄存器传输
- CANID_L：CAN ID的低8位
- CANID_H：CAN ID的高8位，因为我们使用的是标准帧，所以该位一般为0
- 0xAA：代表保存寄存器指令
- RID：表示电机寄存器
- xx：不需要关心这部分数据的多少，可以发长度为4的读取指令，也可以发长度为8的，这部分可要可不要。

2. 反馈报文格式说明

报文ID	D[0]	D[1]	D[2]	D[3]
------	------	------	------	------

MST_ID	CANID_L	CANID_H	0xAA	xx
--------	---------	---------	------	----

- MST_ID：电机内部设置的 **master id** 默认为0
- CANID_L：CAN ID的低8位
- CANID_H：CAN ID的高8位，因为我们使用的是标准帧，所以该位一般为0
- 0xAA：代表保存寄存器指令

⚠ 注意：

- 1.存储参数只在**失能**模式下生效。
- 2.存储参数时将一次性保留全部参数。
- 3.该操作将参数写入片内flash中，每次操作时间最大为30ms，请注意留足足够的时间。
4. flash擦写次数约1万次，请不要频繁发送“存储参数”指令。

3. 控制报文API

Code block

```

1  // 发送保存电机参数命令
2  void save_motor_data(uint16_t id, uint8_t rid)
3  {
4      uint8_t can_id_l = id & 0xFF;          // 低 8 位
5      uint8_t can_id_h = (id >> 8) & 0x07; // 高 3 位
6      uint8_t data[4] = {can_id_l, can_id_h, 0xAA, 0x01};
7      fdcanx_send_data(&hfdcan1, 0x7FF, data, 4);
8  }
```

4. 寄存器参数结构体

Code block

```

1  typedef struct
2  {
3      uint8_t read_flag;          // 读标志
4      uint8_t write_flag;        // 写标志
5      uint8_t save_flag;         // 保存标志
6
7      float UV_Value;            // 低压保护值
8      float KT_Value;           // 扭矩系数
9      float OT_Value;           // 过温保护值
10     float OC_Value;            // 过流保护值

```

```

11     float ACC;           // 加速度
12     float DEC;           // 减速度
13     float MAX_SPD;       // 最大速度
14     uint32_t MST_ID;     // 反馈ID
15     uint32_t ESC_ID;     // 接收ID
16     uint32_t TIMEOUT;    // 超时警报时间
17     uint32_t cmode;      // 控制模式
18     float Damp;          // 电机粘滞系数
19     float Inertia;       // 电机转动惯量
20     uint32_t hw_ver;     // 保留
21     uint32_t sw_ver;     // 软件版本号
22     uint32_t SN;         // 保留
23     uint32_t NPP;        // 电机极对数
24     float Rs;            // 电阻
25     float Ls;            // 电感
26     float Flux;          // 磁链
27     float Gr;            // 齿轮减速比
28     float PMAX;          // 位置映射范围
29     float VMAX;          // 速度映射范围
30     float TMAX;          // 扭矩映射范围
31     float I_BW;          // 电流环控制带宽
32     float KP_ASR;        // 速度环Kp
33     float KI_ASR;        // 速度环Ki
34     float KP_APR;        // 位置环Kp
35     float KI_APR;        // 位置环Ki
36     float OV_Value;      // 过压保护值
37     float GREF;          // 齿轮力矩效率
38     float Deta;          // 速度环阻尼系数
39     float V_BW;          // 速度环滤波带宽
40     float IQ_cl;         // 电流环增强系数
41     float VL_cl;         // 速度环增强系数
42     uint32_t can_br;     // CAN波特率代码
43     uint32_t sub_ver;    // 子版本号
44     float u_off;         // u相偏置
45     float v_off;         // v相偏置
46     float k1;            // 补偿因子1
47     float k2;            // 补偿因子2
48     float m_off;         // 角度偏移
49     float dir;           // 方向
50     float p_m;           // 电机位置
51     float x_out;         // 输出轴位置
52 } esc_inf_t;

```

9. 电机控制实战

9.1 MIT模式控制

1. MIT模式使用的参数: pos_set, vel_set, tor_set, kp_set, kd_set

这里只演示MIT模式的速度控制

2. 在main函数里面调用并下载代码

Code block

```
1  /* 在 dm_motor_ctrl.c 文件中 */
2  void dm_motor_init(void)
3  {
4      memset(&motor[Motor1], 0, sizeof(motor[Motor1]));    // 初始化Motor1电机结构
      体
5
6      // 设置Motor1的电机信息
7      motor[Motor1].id = 0x01;
8      motor[Motor1].mst_id = 0x00;    // 实际没有用上, 只做标识作用
9      motor[Motor1].tmp.read_flag = 1;
10     motor[Motor1].ctrl.mode = mit_mode;    // 选择 MIT 模式
11     motor[Motor1].ctrl.vel_set = 1.0f;    // 设置速度为 1
12     motor[Motor1].ctrl.pos_set = 0.0f;
13     motor[Motor1].ctrl.tor_set = 0.0f;
14     motor[Motor1].ctrl.cur_set = 0.0f;
15     motor[Motor1].ctrl.kp_set = 0.0f;
16     motor[Motor1].ctrl.kd_set = 1.0f;    // 设置 Kd 为 1
17     // DM4310的 PMAX VMAX TMAX
18     motor[Motor1].tmp.PMAX = 12.5f;
19     motor[Motor1].tmp.VMAX = 30.0f;
20     motor[Motor1].tmp.TMAX = 10.0f;
21 }
22
23 /* 在 main.c 文件中*/
24 int main(void)
25 {
26     /* 截取添加的代码如下 */
27     power(1);    // 打开开发板的可控电源接口
28     HAL_Delay(1000);    // 等待电机上电自检通过
29
30     bsp_fdcan_set_baud(&hfdcan1, CAN_CLASS, CAN_BR_1M);    // 设置CAN为经典模
      式 波特率为1M
31     bsp_can_init();
32     dm_motor_init();
33     HAL_Delay(10);
34     /* 往电机的寄存器里面写入MIT模式 */
35     write_motor_data(motor[Motor1].id, 10, mit_mode, 0, 0, 0);    // 修改电机为
      MIT模式
```

```

36     HAL_Delay(100);
37     save_motor_data(motor[Motor1].id, 10);           // 保存电机参数
38     HAL_Delay(100);
39     dm_motor_enable(&hfdcan1, &motor[Motor1]);      // 使能电机
40     HAL_Delay(1000);
41
42     HAL_TIM_Base_Start_IT(&htim3);                  // 开启1ms定时器中断用于发送can
命令
43     while (1)
44     {
45
46     }
47 }
48
49 // 1ms 定时器中断函数
50 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
51 {
52     if (htim->Instance == TIM3) {
53         dm_motor_ctrl_send(&hfdcan1, &motor[Motor1]);
54     }
55 }
56
57 /* 在 dm_motor_ctrl.c 文件中 */
58 // 接收中断回调 处理电机回传的信息，默认master id为0，所以这里 rec_id = 0
59 void fdcan1_rx_callback(void)
60 {
61     uint16_t rec_id;
62     uint8_t rx_data[8] = {0};
63     fdcanx_receive(&hfdcan1, &rec_id, rx_data);
64     switch (rec_id)
65     {
66         case 0x00: dm_motor_fbdata(&motor[Motor1], rx_data);
receive_motor_data(&motor[Motor1], rx_data); break;
67     }
68 }

```



将上面代码下载入单片机，可以观察到电机匀速旋转

⚠ 注意事项：MC开发板最高仅支持24V，禁止接入28V以上电源，过压将导致开发板烧毁；如需使用开发板控制48V电机，请分开供电



3. 打开USB转CAN上位机监测总线数据以及打开 keil debug 窗口观测电机回传的数据信息

USB2CAN

串口号:
COM3

波特率:
921600

数据位:
8

校验位:
None

停止位:
One

序号	时间戳	帧类型	帧格式	状态	数据长度	数据(D0 ~ D7)	ID
1	16:00:06.3640510	标准帧	数据帧	接收	8	01 00 55 0A 01 00 00 00	7FF
2	16:00:06.3640510	标准帧	数据帧	接收	8	01 00 55 0A 01 00 00 00	0
3	16:00:06.5683532	标准帧	数据帧	接收	4	01 00 AA 01	7FF
4	16:00:06.5683532	标准帧	数据帧	接收	4	01 00 AA 01	0
5	16:00:06.6708321	标准帧	数据帧	接收	8	FF FF FF FF FF FF FC	1
6	16:00:06.6708321	标准帧	数据帧	接收	8	11 8C DB 80 07 FF 1E 1C	0
7	16:00:07.6683214	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
8	16:00:07.6683214	标准帧	数据帧	接收	8	11 8C DB 7F F8 00 1E 1D	0
9	16:00:07.6703775	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
10	16:00:07.6703775	标准帧	数据帧	接收	8	11 8C DB 7F F8 04 1E 1D	0
11	16:00:07.6723810	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
12	16:00:07.6723810	标准帧	数据帧	接收	8	11 8C DB 80 08 09 1E 1D	0
13	16:00:07.6723810	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
14	16:00:07.6723810	标准帧	数据帧	接收	8	11 8C DB 80 18 13 1E 1D	0
15	16:00:07.6735816	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
16	16:00:07.6735816	标准帧	数据帧	接收	8	11 8C DB 80 28 1F 1E 1D	0
17	16:00:07.6735816	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
18	16:00:07.6735816	标准帧	数据帧	接收	8	11 8C DE 80 B8 35 1E 1D	0
19	16:00:07.6761630	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
20	16:00:07.6761630	标准帧	数据帧	接收	8	11 8C D0 81 18 3E 1E 1D	0
21	16:00:07.6761630	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
22	16:00:07.6761630	标准帧	数据帧	接收	8	11 8C D2 81 B8 45 1E 1D	0
23	16:00:07.6761630	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
24	16:00:07.6772235	标准帧	数据帧	接收	8	11 8C E2 81 B8 45 1E 1D	0
25	16:00:07.6772235	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1
26	16:00:07.6772235	标准帧	数据帧	接收	8	11 8C E5 82 38 49 1E 1D	0
27	16:00:07.6808097	标准帧	数据帧	接收	8	7F FF 84 30 00 33 37 FF	1

波特率(kbps)
1000

帧ID(HEX)
521

发送数据(HEX)
D0 D1 D2 D3 D4 D5 D6 D7
12 23 34 45 56 67 78 89

发送次数
1

帧格式
标准帧

发送

DATA递增

定时发送

帧类型
数据帧

停止

发送帧数: 0

接收帧数: 1852

Name

Value

Type

motor	0x240001E0	motor	struct <u
motor[0].para	0x240001E4	struct <u	
id	0x00000001	int	
state	0x00000001	int	
p_int	0x0000701A	int	
v_int	0x00000815	int	
t_int	0x00000000	int	
kp_int	0x00000000	int	
kd_int	0x00000000	int	
pos	-1.54936314	float	
vel	0.974359512	float	
tor	0.0952377319	float	
Kp	0	float	
Kd	0	float	
Tmos	30	float	
Tcoil	29	float	
motor[0].ctrl	0x2400021C	struct <u	
mode	0x01	uchar	
pos_set	0	float	
vel_set	1	float	
tor_set	0	float	
cur_set	0	float	
kp_set	0	float	
kd_set	1	float	

<Enter expression>

9.2 位置速度模式控制

1. 位置速度模式使用的参数: pos_set, vel_set
2. 在main函数里面调用并下载代码

Code block

```
1 /* 在 dm_motor_ctrl.c 文件中 */
```



```

2 void dm_motor_init(void)
3 {
4     memset(&motor[Motor1], 0, sizeof(motor[Motor1]));    // 初始化Motor1电机结构
    体
5
6     // 设置Motor1的电机信息
7     motor[Motor1].id = 0x01;
8     motor[Motor1].mst_id = 0x00;    // 实际没有用上, 只做标识作用
9     motor[Motor1].tmp.read_flag = 1;
10    motor[Motor1].ctrl.mode = pos_mode;    // 选择位置速度模式
11    motor[Motor1].ctrl.vel_set = 1.0f;    // 设置速度为 1
12    motor[Motor1].ctrl.pos_set = 3.14f;    // 设置位置为 3.14
13    motor[Motor1].ctrl.tor_set = 0.0f;
14    motor[Motor1].ctrl.cur_set = 0.0f;
15    motor[Motor1].ctrl.kp_set = 0.0f;
16    motor[Motor1].ctrl.kd_set = 0.0f;
17    // DM4310的 PMAX VMAX TMAX
18    motor[Motor1].tmp.PMAX = 12.5f;
19    motor[Motor1].tmp.VMAX = 30.0f;
20    motor[Motor1].tmp.TMAX = 10.0f;
21 }
22
23 /* 在 main.c 文件中*/
24 int main(void)
25 {
26     /* 截取添加的代码如下 */
27     power(1);    // 打开开发板的可控电源接口
28     HAL_Delay(1000);    // 等待电机上电自检通过
29
30     bsp_fdcan_set_baud(&hfdcan1, CAN_CLASS, CAN_BR_1M);    // 设置CAN为经典模
    式 波特率为1M
31     bsp_can_init();
32     dm_motor_init();
33     HAL_Delay(10);
34     /* 往电机的寄存器里面写入位置速度模式 */
35     write_motor_data(motor[Motor1].id, 10, pos_mode, 0, 0, 0);    // 修改电机为位
    置速度模式
36     HAL_Delay(100);
37     save_motor_data(motor[Motor1].id, 10);    // 保存电机参数
38     HAL_Delay(100);
39     dm_motor_enable(&hfdcan1, &motor[Motor1]);    // 使能电机
40     HAL_Delay(1000);
41
42     HAL_TIM_Base_Start_IT(&htim3);    // 开启1ms定时器中断用于发送can
    命令
43     while (1)
44     {

```



```

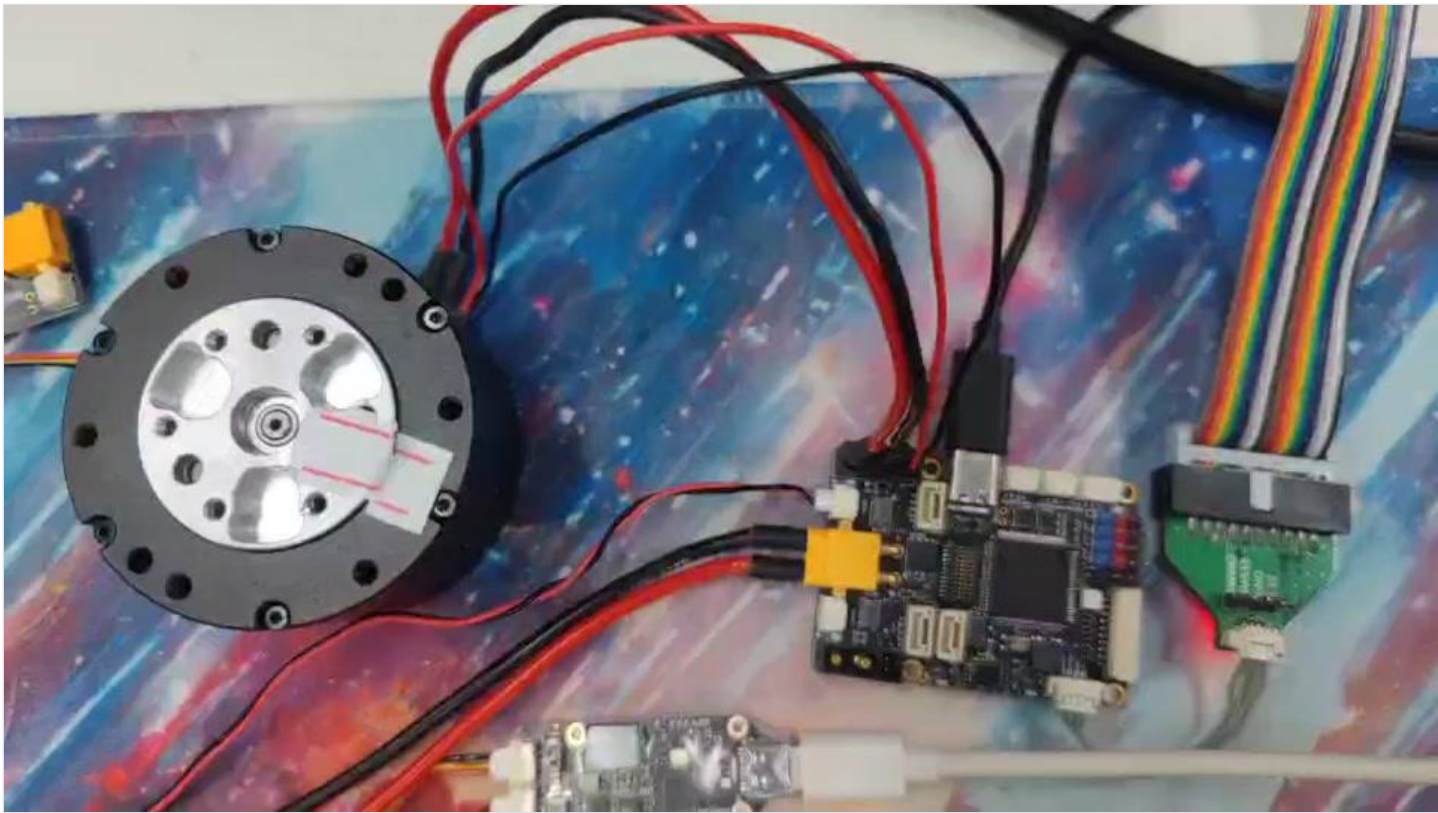
45
46     }
47 }
48
49 // 1ms 定时器中断函数
50 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
51 {
52     if (htim->Instance == TIM3) {
53         dm_motor_ctrl_send(&hfdcan1, &motor[Motor1]);
54     }
55 }
56
57 /* 在 dm_motor_ctrl.c 文件中 */
58 // 接收中断回调 处理电机回传的信息，默认master id为0，所以这里 rec_id = 0
59 void fdcan1_rx_callback(void)
60 {
61     uint16_t rec_id;
62     uint8_t rx_data[8] = {0};
63     fdcanx_receive(&hfdcan1, &rec_id, rx_data);
64     switch (rec_id)
65     {
66         case 0x00: dm_motor_fbdata(&motor[Motor1], rx_data);
67         receive_motor_data(&motor[Motor1], rx_data); break;
68     }
69 }

```



将上面代码下载入单片机，可以观察到电机旋转到的位置

⚠️ 注意事项：MC开发板最高仅支持24V，禁止接入28V以上电源，过压将导致开发板烧毁；如需使用开发板控制48V电机，请分开供电



3. 打开USB转CAN上位机监测总线数据以及打开 keil debug 窗口观测电机回传的数据信息

USB2CAN

串口号:
COM3

波特率:
921600

数据位:
8

校验位:
None

停止位:
One

序号	时间戳	轴类型	轴格式	状态	数据长度	数据(D0 - D7)	ID
1	16:25:33.8519471	标准轴	数据轴	接收	8	01 00 55 0A 02 00 00 00	7FF
2	16:25:33.8519471	标准轴	数据轴	接收	8	01 00 55 0A 02 00 00 00	0
3	16:25:34.0542871	标准轴	数据轴	接收	4	01 00 AA 01	7FF
4	16:25:34.0542871	标准轴	数据轴	接收	4	01 00 AA 01	0
5	16:25:34.1551129	标准轴	数据轴	接收	8	FF FF FF FF FF FF FC	101
6	16:25:34.1551129	标准轴	数据轴	接收	8	11 91 22 7F F8 04 1E 1E	0
7	16:25:35.1566699	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
8	16:25:35.1566699	标准轴	数据轴	接收	8	11 91 23 7F F8 03 1E 1E	0
9	16:25:35.1566699	标准轴	数据轴	接收	8	11 91 23 80 08 02 1E 1E	0
10	16:25:35.1566699	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
11	16:25:35.1566699	标准轴	数据轴	接收	8	11 91 23 7F F8 04 1E 1E	0
12	16:25:35.1566699	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
13	16:25:35.1566699	标准轴	数据轴	接收	8	11 91 23 7F F8 03 1E 1E	0
14	16:25:35.1566699	标准轴	数据轴	接收	8	11 91 23 7F F8 04 1E 1E	0
15	16:25:35.1606666	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
16	16:25:35.1606666	标准轴	数据轴	接收	8	11 91 23 7F F8 04 1E 1E	0
17	16:25:35.1606666	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
18	16:25:35.1606666	标准轴	数据轴	接收	8	11 91 23 7F F8 04 1E 1E	0
19	16:25:35.1626632	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
20	16:25:35.1626632	标准轴	数据轴	接收	8	11 91 23 7F F8 04 1E 1E	0
21	16:25:35.1626632	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
22	16:25:35.1626632	标准轴	数据轴	接收	8	11 91 23 7F F8 05 1E 1E	0
23	16:25:35.1646839	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
24	16:25:35.1646839	标准轴	数据轴	接收	8	11 91 23 80 08 09 1E 1E	0
25	16:25:35.1646839	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101
26	16:25:35.1646839	标准轴	数据轴	接收	8	11 91 23 80 18 09 1E 1E	0
27	16:25:35.1666598	标准轴	数据轴	接收	8	C3 F5 48 40 00 00 80 3F	101

波特率(kbps)
1000

帧ID(HEX)
521

发送数据(HEX)
D0 D1 D2 D3 D4 D5 D6 D7
12 23 34 45 56 67 78 89

发送次数
1

帧格式
标准帧

发送

停止

Disconnect

0 FPS

2000 FPS

0.0%

发送帧数: 0

接收帧数: 6116

name

value

type

motor

0x240001E0

motor

motor[0].para

0x240001E4

struct <ui

id

0x00000001

int

state

0x00000001

int

p_int

0x0000007F

int

v_int

0x0000007F

int

t_int

0x0000007E9

int

kp_int

0x00000000

int

kd_int

0x00000000

int

pos

3.13973427

float

vel

-0.0073261261

float

tor

-0.11477375

float

Kp

0

float

Kd

0

float

Tmos

31

float

Tcoil

30

float

motor[0].ctrl

0x2400021C

struct <ui

mode

0x02

uchar

pos_set

3.1400001

float

vel_set

1

float

tor_set

0

float

cur_set

0

float

kp_set

0

float

kd_set

0

float

<Enter expression>

9.3 速度模式控制

- 速度模式使用的参数：`vel_set`
- 在main函数里面调用并下载代码

Code block

```

1  /* 在 dm_motor_ctrl.c 文件中 */
2  void dm_motor_init(void)
3  {
4      memset(&motor[Motor1], 0, sizeof(motor[Motor1]));    // 初始化Motor1电机结构
      体
5
6      // 设置Motor1的电机信息
7      motor[Motor1].id = 0x01;
8      motor[Motor1].mst_id = 0x00;    // 实际没有用上，只做标识作用
9      motor[Motor1].tmp.read_flag = 1;
10     motor[Motor1].ctrl.mode = spd_mode;    // 选择位置速度模式
11     motor[Motor1].ctrl.vel_set = 2.0f;    // 设置速度为 2
12     motor[Motor1].ctrl.pos_set = 0.0f;
13     motor[Motor1].ctrl.tor_set = 0.0f;
14     motor[Motor1].ctrl.cur_set = 0.0f;
15     motor[Motor1].ctrl.kp_set = 0.0f;
16     motor[Motor1].ctrl.kd_set = 0.0f;
17     // DM4310的 PMAX VMAX TMAX
18     motor[Motor1].tmp.PMAX = 12.5f;
19     motor[Motor1].tmp.VMAX = 30.0f;
20     motor[Motor1].tmp.TMAX = 10.0f;
21 }
22
23 /* 在 main.c 文件中*/
24 int main(void)
25 {
26     /* 截取添加的代码如下 */
27     power(1);    // 打开开发板的可控电源接口
28     HAL_Delay(1000);    // 等待电机上电自检通过
29
30     bsp_fdcan_set_baud(&hfdcan1, CAN_CLASS, CAN_BR_1M);    // 设置CAN为经典模
      式 波特率为1M
31     bsp_can_init();
32     dm_motor_init();
33     HAL_Delay(10);
34     /* 往电机的寄存器里面写入位置速度模式 */
35     write_motor_data(motor[Motor1].id, 10, spd_mode, 0, 0, 0);    // 修改电机为速
      度模式
36     HAL_Delay(100);
37     save_motor_data(motor[Motor1].id, 10);    // 保存电机参数
38     HAL_Delay(100);
39     dm_motor_enable(&hfdcan1, &motor[Motor1]);    // 使能电机
40     HAL_Delay(1000);
41
42     HAL_TIM_Base_Start_IT(&htim3);    // 开启1ms定时器中断用于发送can
      命令
43     while (1)

```

```

44     {
45
46     }
47 }
48
49 // 1ms 定时器中断函数
50 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
51 {
52     if (htim->Instance == TIM3) {
53         dm_motor_ctrl_send(&hfdcan1, &motor[Motor1]);
54     }
55 }
56
57 /* 在 dm_motor_ctrl.c 文件中 */
58 // 接收中断回调 处理电机回传的信息，默认master id为0，所以这里 rec_id = 0
59 void fdcan1_rx_callback(void)
60 {
61     uint16_t rec_id;
62     uint8_t rx_data[8] = {0};
63     fdcanx_receive(&hfdcan1, &rec_id, rx_data);
64     switch (rec_id)
65     {
66         case 0x00: dm_motor_fbdata(&motor[Motor1], rx_data);
67         receive_motor_data(&motor[Motor1], rx_data); break;
68     }
69 }

```



将上面代码下载入单片机，可以观察到电机按指定速度旋转

⚠️ 注意事项：MC开发板最高仅支持24V，禁止接入28V以上电源，过压将导致开发板烧毁；如需使用开发板控制48V电机，请分开供电


```

1  /* 在 dm_motor_ctrl.c 文件中 */
2  void dm_motor_init(void)
3  {
4      memset(&motor[Motor1], 0, sizeof(motor[Motor1]));    // 初始化Motor1电机结构
      体
5
6      // 设置Motor1的电机信息
7      motor[Motor1].id = 0x01;
8      motor[Motor1].mst_id = 0x00;    // 实际没有用上，只做标识作用
9      motor[Motor1].tmp.read_flag = 1;
10     motor[Motor1].ctrl.mode = psi_mode;    // 选择位置速度模式
11     motor[Motor1].ctrl.vel_set = 1.0f;    // 设置速度为 1
12     motor[Motor1].ctrl.pos_set = 6.28f;    // 设置位置为 6.28
13     motor[Motor1].ctrl.tor_set = 0.0f;
14     motor[Motor1].ctrl.cur_set = 0.03f;    // 设置电流为0.03，该值为标么值
15     motor[Motor1].ctrl.kp_set = 0.0f;
16     motor[Motor1].ctrl.kd_set = 0.0f;
17     // DM4310的 PMAX VMAX TMAX
18     motor[Motor1].tmp.PMAX = 12.5f;
19     motor[Motor1].tmp.VMAX = 30.0f;
20     motor[Motor1].tmp.TMAX = 10.0f;
21 }
22
23 /* 在 main.c 文件中*/
24 int main(void)
25 {
26     /* 截取添加的代码如下 */
27     power(1);    // 打开开发板的可控电源接口
28     HAL_Delay(1000);    // 等待电机上电自检通过
29
30     bsp_fdcan_set_baud(&hfdcan1, CAN_CLASS, CAN_BR_1M);    // 设置CAN为经典模
      式 波特率为1M
31     bsp_can_init();
32     dm_motor_init();
33     HAL_Delay(10);
34     /* 往电机的寄存器里面写入位置速度模式 */
35     write_motor_data(motor[Motor1].id, 10, psi_mode, 0, 0, 0);    // 修改电机为力
      位混控模式
36     HAL_Delay(100);
37     save_motor_data(motor[Motor1].id, 10);    // 保存电机参数
38     HAL_Delay(100);
39     dm_motor_enable(&hfdcan1, &motor[Motor1]);    // 使能电机
40     HAL_Delay(1000);
41
42     HAL_TIM_Base_Start_IT(&htim3);    // 开启1ms定时器中断用于发送can
      命令
43     while (1)

```

```

44     {
45
46     }
47 }
48
49 // 1ms 定时器中断函数
50 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
51 {
52     if (htim->Instance == TIM3) {
53         dm_motor_ctrl_send(&hfdcan1, &motor[Motor1]);
54     }
55 }
56
57 /* 在 dm_motor_ctrl.c 文件中 */
58 // 接收中断回调 处理电机回传的信息，默认master id为0，所以这里 rec_id = 0
59 void fdcan1_rx_callback(void)
60 {
61     uint16_t rec_id;
62     uint8_t rx_data[8] = {0};
63     fdcanx_receive(&hfdcan1, &rec_id, rx_data);
64     switch (rec_id)
65     {
66         case 0x00: dm_motor_fbdata(&motor[Motor1], rx_data);
67         receive_motor_data(&motor[Motor1], rx_data); break;
68     }
69 }

```



将上面代码下载入单片机，可以观察到电机旋转会到设定的位置，并且如果电流设定比较小的话，用手可以捏停电机

⚠️ 注意事项：MC开发板最高仅支持24V，禁止接入28V以上电源，过压将导致开发板烧毁；如需使用开发板控制48V电机，请分开供电



3. 打开USB转CAN上位机监测总线数据以及打开 keil debug 窗口观测电机回传的数据信息

USB2CAN

串口号:
COM3

波特率:
921600

数据位:
8

校验位:
None

停止位:
One

CAN

UART

CAN负载

序号	时间戳	轴类型	轴格式	状态	数据长度	数据(D0 - D7)	ID
1	17:03:53.3833721	标准轴	数据轴	接收	8	01 00 55 0A 04 00 00 00	7FF
2	17:03:53.3833721	标准轴	数据轴	接收	8	01 00 55 0A 04 00 00 00	0
3	17:03:53.5864220	标准轴	数据轴	接收	4	01 00 AA 01	7FF
4	17:03:53.5864220	标准轴	数据轴	接收	4	01 00 AA 01	0
5	17:03:53.6885380	标准轴	数据轴	接收	8	FF FF FF FF FF FF FC	301
6	17:03:53.6885380	标准轴	数据轴	接收	8	11 67 78 7F 7F 1F 1D	0
7	17:03:54.6877010	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
8	17:03:54.6877010	标准轴	数据轴	接收	8	11 67 78 7F E8 00 1F 1E	0
9	17:03:54.6896617	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
10	17:03:54.6896617	标准轴	数据轴	接收	8	11 67 78 7F F8 00 1F 1E	0
11	17:03:54.6896617	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
12	17:03:54.6896617	标准轴	数据轴	接收	8	11 67 78 7F F8 00 1F 1E	0
13	17:03:54.6916659	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
14	17:03:54.6916659	标准轴	数据轴	接收	8	11 67 78 7F F8 01 1F 1E	0
15	17:03:54.6916659	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
16	17:03:54.6916659	标准轴	数据轴	接收	8	11 67 78 7F F8 00 1F 1E	0
17	17:03:54.6936626	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
18	17:03:54.6936626	标准轴	数据轴	接收	8	11 67 78 7F F8 00 1F 1E	0
19	17:03:54.6936626	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
20	17:03:54.6936626	标准轴	数据轴	接收	8	11 67 78 80 08 02 1F 1E	0
21	17:03:54.6956586	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
22	17:03:54.6956586	标准轴	数据轴	接收	8	11 67 78 7F F8 03 1F 1E	0
23	17:03:54.6956586	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
24	17:03:54.6956586	标准轴	数据轴	接收	8	11 67 78 80 08 05 1F 1E	0
25	17:03:54.6976623	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301
26	17:03:54.6976623	标准轴	数据轴	接收	8	11 67 78 80 18 06 1F 1E	0
27	17:03:54.6976623	标准轴	数据轴	接收	8	C3 F5 C8 40 64 00 C8 00	301

波特率(kbps):
1000

帧ID(HEX):
521

发送数据(HEX):
D0 D1 D2 D3 D4 D5 D6 D7
12 23 34 45 56 67 78 89

发送次数:
1

帧格式:
标准帧

发送

ID递增:
1

帧类型:
数据帧

停止

Disconnect

0 FPS

1996 FPS

0.0%

发送帧数: 0

接收帧数: 2245

name

value

type

motor

0x240001E0 motor

struct <u>

motor[0].para

0x240001E4

struct <u>

id

0x00000001

int

state

0x0000BECC

int

p_int

0x0000083F

int

v_int

0x0000C80F

int

t_int

0x00000000

int

kp_int

0x00000000

int

kd_int

0x00000000

int

pos

6.13660622

float

vel

0.915750504

float

tor

0.0757017136

float

Kp

0

float

Kd

0

float

Tmos

31

float

Tcoil

30

float

motor[0].ctrl

0x2400021C

struct <u>

mode

0x04

uchar

pos_set

6.28000021

float

vel_set

1

float

tor_set

0

float

cur_set

0.0199999996

float

kp_set

0

float

kd_set

0

float

<Enter expression>

9.5 寄存器参数读取

1. 在main函数里面调用并下载代码

Code block

```
1 /* 在 dm_motor_ctrl.c 文件中 */
2 void dm_motor_init(void)
```



```

3  {
4      memset(&motor[Motor1], 0, sizeof(motor[Motor1]));    // 初始化Motor1电机结构
   体
5
6      // 设置Motor1的电机信息
7      motor[Motor1].id = 0x01;
8      motor[Motor1].mst_id = 0x00;    // 实际没有用上, 只做标识作用
9      motor[Motor1].tmp.read_flag = 1;    // 将读取flag设为1, 读取完毕后会设置为0
10 }
11
12 /* 在 main.c 文件中*/
13 int main(void)
14 {
15     /* 截取添加的代码如下 */
16     power(1);    // 打开开发板的可控电源接口
17     HAL_Delay(1000); // 等待电机上电自检通过
18
19     bsp_fdcan_set_baud(&hfdcan1, CAN_CLASS, CAN_BR_1M);    // 设置CAN为经典模
   式 波特率为1M
20     bsp_can_init();
21     dm_motor_init();
22     HAL_Delay(10);
23
24     HAL_TIM_Base_Start_IT(&htim3);    // 开启1ms定时器中断用于发送can
   命令
25     while (1)
26     {
27
28     }
29 }
30
31 // 1ms 定时器中断函数
32 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
33 {
34     if (htim->Instance == TIM3) {
35         read_all_motor_data(&motor[Motor1]);
36     }
37 }
38
39 /* 在 dm_motor_ctrl.c 文件中 */
40 // 接收中断回调 处理电机回传的信息, 默认master id为0, 所以这里 rec_id = 0
41 void fdcan1_rx_callback(void)
42 {
43     uint16_t rec_id;
44     uint8_t rx_data[8] = {0};
45     fdcanx_receive(&hfdcan1, &rec_id, rx_data);
46     switch (rec_id)

```

```

47     {
48         case 0x00: dm_motor_fbdata(&motor[Motor1], rx_data);
        receive_motor_data(&motor[Motor1], rx_data); break;
49     }
50 }

```

2. 打开USB转CAN上位机监测总线数据以及打开 keil debug 窗口观测电机回传的数据信息

发送的读取寄存器命令

序号	时间戳	帧类型	帧格式	状态	数据长度	数据(D0 - D7)	ID
1	17:15:46.8870170	标准帧	数据帧	接收	4	01 00 33 00	7FF
2	17:15:46.8870170	标准帧	数据帧	接收	8	01 00 33 00 00 00 70 41	0
3	17:15:46.8890403	标准帧	数据帧	接收	4	01 00 33 01	7FF
4	17:15:46.8890403	标准帧	数据帧	接收	8	01 00 33 01 00 00 00 00	0
5	17:15:46.8890403	标准帧	数据帧	接收	4	01 00 33 02	7FF
6	17:15:46.8890403	标准帧	数据帧	接收	8	01 00 33 02 00 00 F0 42	0
7	17:15:46.8890403	标准帧	数据帧	接收	4	01 00 33 03	7FF
8	17:15:46.8910419	标准帧	数据帧	接收	8	01 00 33 03 48 E1 7A 3F	0
9	17:15:46.8910419	标准帧	数据帧	接收	4	01 00 33 04	7FF
10	17:15:46.8910419	标准帧	数据帧	接收	8	01 00 33 04 00 00 00 40	0
11	17:15:46.8930327	标准帧	数据帧	接收	4	01 00 33 05	7FF
12	17:15:46.8930327	标准帧	数据帧	接收	8	01 00 33 05 00 00 00 C0	0
13	17:15:46.8930327	标准帧	数据帧	接收	4	01 00 33 06	7FF
14	17:15:46.8930327	标准帧	数据帧	接收	8	01 00 33 06 00 00 16 44	0
15	17:15:46.8950242	标准帧	数据帧	接收	4	01 00 33 07	7FF
16	17:15:46.8950242	标准帧	数据帧	接收	8	01 00 33 07 00 00 00 00	0
17	17:15:46.8950242	标准帧	数据帧	接收	4	01 00 33 08	7FF
18	17:15:46.8950242	标准帧	数据帧	接收	8	01 00 33 08 01 00 00 00	0
19	17:15:46.8970175	标准帧	数据帧	接收	4	01 00 33 09	7FF
20	17:15:46.8970175	标准帧	数据帧	接收	8	01 00 33 09 00 00 00 00	0
21	17:15:46.8970175	标准帧	数据帧	接收	4	01 00 33 0A	7FF
22	17:15:46.8970175	标准帧	数据帧	接收	8	01 00 33 0A 04 00 00 00	0
23	17:15:46.8990213	标准帧	数据帧	接收	4	01 00 33 0B	7FF
24	17:15:46.8990213	标准帧	数据帧	接收	8	01 00 33 0B 1E 2A B6 38	0
25	17:15:46.8990213	标准帧	数据帧	接收	4	01 00 33 0C	7FF
26	17:15:46.8990213	标准帧	数据帧	接收	8	01 00 33 0C 99 3D B5 37	0
27	17:15:46.9010116	标准帧	数据帧	接收	4	01 00 33 0D	7FF

读取到的电机寄存器信息

Name	Value	Type
motor[0].tmp	0x24000238	struct
read_flag	0x00	uchar
write_flag	0x00	uchar
save_flag	0x00	uchar
UV_Value	15	float
KT_Value	0	float
OT_Value	120	float
OC_Value	0.980000019	float
ACC	2	float
DEC	-2	float
MAX_SPD	600	float
MST_ID	0x00000000	uint
ESC_ID	0x00000001	uint
TIMEOUT	0x00000000	uint
cmode	0x00000000	uint
Damp	8.722044e-05	float
Inertia	2.16055651e-05	float
hw_ver	0x56303033	uint
sw_ver	0x35313335	uint
SN	0x54303035	uint
NPP	0x00000000	uint
Rs	0.371584713	float
Ls	0.000134322618	float
Flux	0.00323708006	float
Gr	10	float
PMAX	12.5	float
VMAX	30	float
TMAX	10	float
I_BW	1000	float
KP_ASR	0.00100000005	float
KI_ASR	0.00200000009	float
KP_APR	54	float
KI_APR	0	float
OV_Value	60	float
GREF	1	float
Deta	4	float
V_BW	40	float
IQ_cl	2500	float
VL_cl	100	float
can_br	0x00000004	uint
sub_ver	0x00000033	uint
u_off	1978.5	float
v_off	1982.5	float
k1	1.01791477	float
k2	-1.57245898	float
m_off	0	float
dir	2	float
p_m	-0.000830962032	float
x_out	-0.262310714	float

10. 关于DM-MC01 F4的代码



F4跟H7的代码在使用上没有什么区别，关于电机控制的应用层代码是一样的，有区别的就是F4没有办法使用FDCAN以及 **bsp_can** 的底层文件不一样，用户可以自行参考每个文件，方便进行移植，只要一开始打通CAN的通信，就可以进行控制代码的调试，具体步骤都可以参考上面的方法

11. 参考资料

上面演示所使用的例程链接：[STM32 H7 控制电机例程](#)

达妙协议文档说明：[📄 调试助手使用说明书（达妙驱动控制协议） V1.4.pdf](#)

ST 官方的应用笔记：[📄 CAN FD应用笔记.pdf](#)

FDCAN 更详细的介绍推荐安富莱的教程：[📄 安富莱_STM32-V7开发板_用户手册，含BSP驱动包设计 \(V3.5\) .pdf](#)
